UNIVERSITAT POLITÈCNICA DE VALÈNCIA

CODISEÑO HARDWARE-SOFTWARE DE SISTEMAS INTEGRADOS BASADOS EN NIOS II

DISEÑO DE UN HARDWARE DE REFERENCIA PARA DE1-SOC

Ricardo José Colom Palero

Valencia, 21 de enero de 2026

Versión 17.1, Quartus Prime, DE1-SoC

CONTENIDOS

1. Objetivos **3**

2. Introducción **3**

3. Sistema Nios II **5**

4. El Reloj del Sistema **7**

4.1. El Reloj y el Interfaz SDRAM 7

4.2. Propuesta de la Estructura de Relojes del Sistema 8

5. Cache de Datos en Nios II **8**

6. Entorno de Visualización **11**

6.1. Utilizando un monitor VGA 11

7. Pasos a seguir en la realización de un sistema utilizando la Platform Designer **14**

8. Integración del Sistema Nios II en el proyecto de Quartus Prime **43**

8.1. Instanciación del Módulo Generado por el *Platform Designer* 43

8.2. Asignación de Pines 47

8.3. Compilación del Proyecto de Quartus Prime 47

8.4. Descargar el Diseño Hardware en la FPGA 47

9. Desarrollo y Ejecución del Programa de Aplicación **48**

9.1. Crear un Proyecto para una Nueva Aplicación C/C++ 48

9.2. Compilación del Proyecto 50

9.3. Ejecución del Programa 51

9.4. Cuestiones sobre el ejercicio 52

10. Ejercicios Prácticos **52**

10.1. Ejercicio 1 – El manejo de puertos paralelos 52

10.2. Ejercicio 2 – El manejo de las Interrupciones 53

10.3. Ejercicio 3 – Visualización en la pantalla VGA 53

10.4. Ejercicio 4 – Diseño de un reloj 55

10.5. Tarea 1 – Entregables 56

1. Objetivos

La finalidad de este ejercicio es que el alumno realice el diseño de un sistema integrado completo basado en el procesador Nios II de Altera, que pueda utilizar como base y referencia para sucesivos ejercicios. Con ello, se intenta evitar que en cada ejercicio haya que realizar de nuevo el diseño de la parte hardware del sistema. Como mucho se realizará la incorporación de algún elemento nuevo. El sistema integrado que se planteará en este ejercicio se particularizara para ser utilizado en la placa DE1-SoC de Altera, por ello se pretenderá que englobe el mayor número de elementos de dicha placa.

2. Introducción

En este ejercicio el alumno desarrollará e implementará un sistema integrado basado en Nios II utilizando la FPGA Cyclone V de la placa DE1-SoC de Altera. La herramienta de Quartus Prime junto con la de la *Platform Designer* más el entorno de desarrollo software Nios II SBT, permitirán alcanzar los objetivos planteados en el ejercicio.

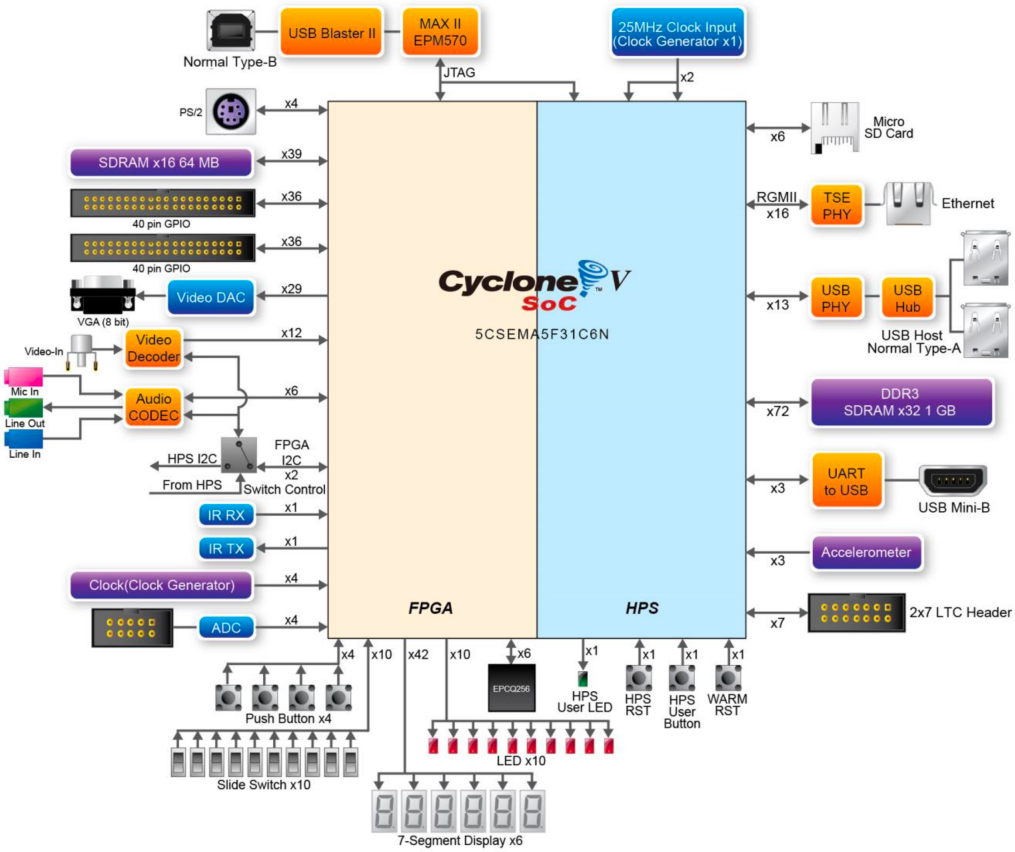

Figura 1. Diagrama de bloques de los componentes hardware de la placa DE1-SoC de Altera.

El procesador Nios II es un procesador soft-core, definido en un lenguaje de descripción hardware (como Verilog o VHDL), el cual puede ser implementado en una FPGA de Altera utilizando las herramientas de sistemas de Quartus Prime. Para implementar un sistema integrado completo, además del procesador, se necesitan otras unidades funcionales como las memorias, interfaces de entrada y salida, temporizadores, interfaces de comunicación, etc. Todo el sistema se emplazará dentro de la FPGA utilizando los diferentes recursos, elementos lógicos, memorias, unidades aritméticas, etc. Se va a realizar un sistema basado en el procesador Nios, para la placa DE1-SoC de Altera. En la *Figura 1*, se muestra el diagrama de bloques de la estructura hardware de la placa DE1-SoC, en ella se puede apreciar el gran número de elementos que contiene. Cabe recordar que la FPGA empleada en la placa DE1-

SoC es una Cyclone V, que ya viene provista de una procesador ARM; es decir, un procesador un hard-core. De los elementos periféricos que tiene la placa DE1-SoC hay unos cuantos que solo pueden ser utilizados por el procesador ARM, son los que en la *Figura 1*, se conectan a la parte de la Cyclone V denominada como HPS. El resto de los elementos se conectan a la mitad denominada FPGA, por tanto son accesibles por medio de los elementos lógicos que contiene la Cyclone V, así que serán los periféricos que utilizará el procesador Nios II que se va a emplear.

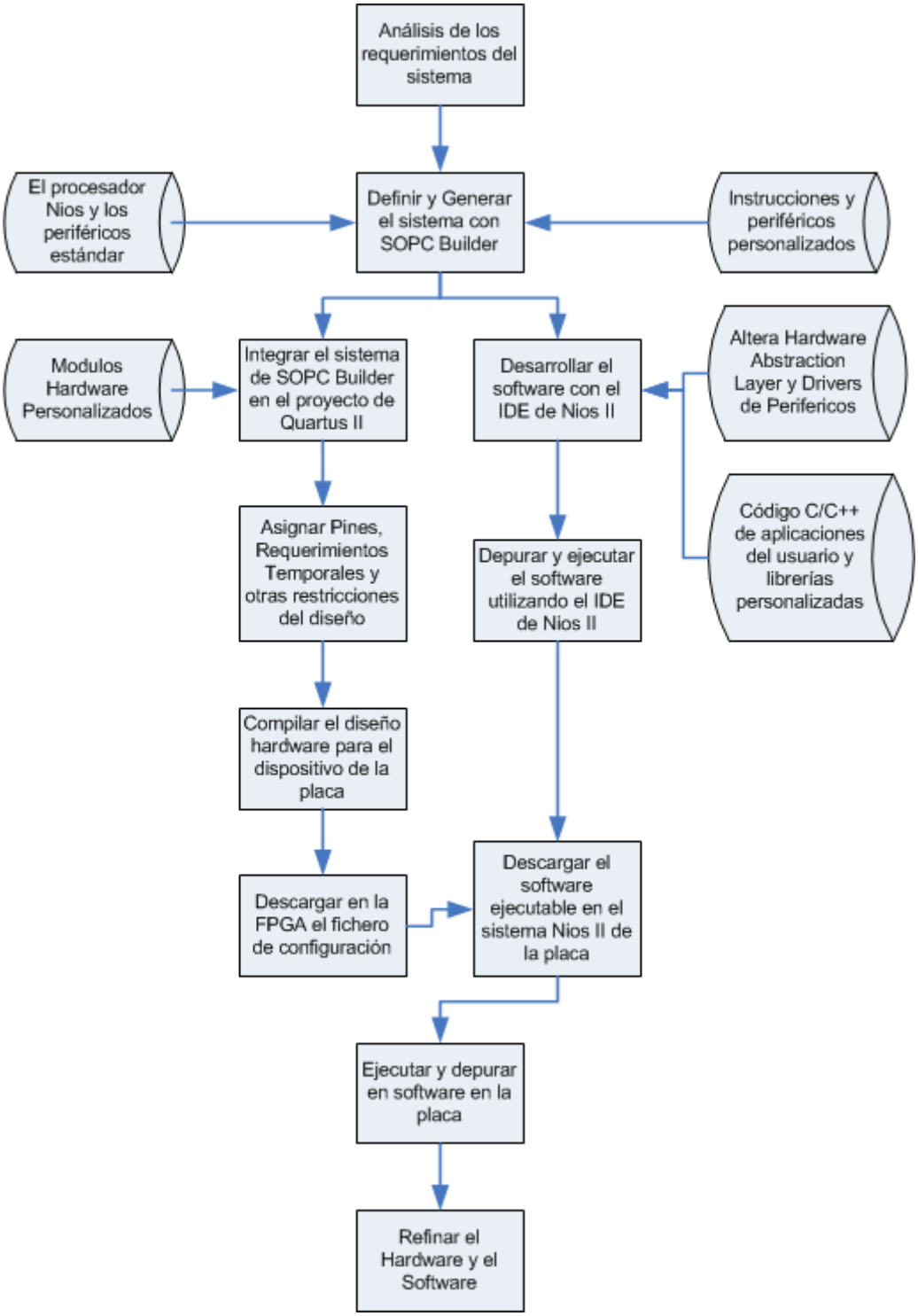

Figura 2. Flujo de diseño de un sistema Nios II.

En este ejercicio, el flujo de desarrollo del sistema muestra mediante instrucciones paso a paso cómo usar la *Platform Designer* junto con el Quartus Prime para implementar un sistema integrado. El último paso en el proceso de desarrollo consiste en configurar el circuito diseñado en la FPGA y ejecutar un programa de aplicación. Las pantallas capturadas en el tutorial se han obtenido utilizando la versión 17.1 del programa Quartus Prime de Altera.

3. Sistema Nios II

En esta sección se explica el flujo de diseño para crear un sistema con Nios II y su verificación en una placa de pruebas. La *Figura 2*, muestra el flujo de diseño completo, consistente en tres etapas: el diseño del hardware, el diseño del software y el diseño del sistema, que involucra tanto al hardware como al software. La realización de un sistema Nios sencillo, puede ser hecha por una única persona. Los sistemas complejos pueden requerir de un equipo de trabajo donde haya un responsable de la etapa hardware y otro del software. En este último caso conviene clarificar que ficheros o que datos deben pasarse entre los diferentes equipos.

Un sistema integrado basado en Nios II, como el que será implementado por medio de este ejercicio para la placa DE1-SoC, se muestra en la *Figura 3*. El procesador Nios II y los elementos necesarios para conectarse con otros circuitos integrados de la placa DE1-SoC se implementan en la FPGA Cyclone V. Estos componentes se conectan entre ellos a través de una red de interconexión denominada “System Interconnect Fabric”. La memoria SDRAM que dispone la placa DE1-SoC se pueden utilizar a través de la oportuna interface. Las interfaces de entrada y salida paralelas y serie permiten a los sistemas integrados disponer de puertos de entrada salida. Es necesario utilizar una interface especial JTAG UART, para interconectar la circuitería de la placa DE1-SoC con el PC por medio de un enlace USB (Universal Serial Bus). Esta circuitería y su software asociado se conocen como USB-Blaster. Otro módulo denominado JTAG Debug permite al PC controlar el sistema Nios II, permitiendo realizar operaciones de carga del programa en memoria, comienzo y paro de la ejecución de un programa, establecer puntos de ruptura o capturar señales en tiempo real para realizar una verificación del sistema. Todos los elementos del sistema Nios II implementados en la FPGA se definen utilizando un lenguaje de descripción hardware. La herramienta *Platform Designer* se utiliza para implementar el sistema integrado que se desea, eligiendo los componentes necesarios, estableciendo los parámetros de cada uno de los componentes y los requerimientos generales del sistema. En este ejercicio se mostrará la capacidad de la *Platform Designer* para diseñar un sistema integrado como el de la *Figura 3*. El mismo proceso se puede llevar a cabo para la realización de sistemas más complejos.

El sistema ejemplo mostrado en la *Figura 3*, pretende ser un sistema integrado de referencia para ser utilizando en ejercicios posteriores. Está pensado para ser utilizado en la placa DE1-SoC de Altera, por ello se le ha dotado de un gran número de componentes, con la finalidad de poder realizar unos ejercicios mínimamente adecuados. Se puede decir que el sistema planteado en la *Figura 3* incluye los requerimientos básicos de un diseño. Sin embargo, tal como se ha mostrado en la *Figura 1*, la cantidad de componentes de la placa DE1-SoC es mucho mayor. Tal como se puede deducir de la *Figura 3*, se pretende realizar el diseño de un sistema integrado para DE1-SoC que incluya el mayor número de elementos, por ejemplo:

- Procesador Nios II.
- Interfaz JTAG UART.
- Timer o temporizador.
- Contador o “Performance Counter”.
- Identificador del Sistema o “System ID”.
- 10 interruptores de palanca.

- 4 pulsadores.
- 10 diodos LED rojos.
- 6 displays 7-segmentos.
- Controlador para memoria SDRAM.
- Controlador del códec de Audio.
- Controlador de teclado y/o ratón PS2.

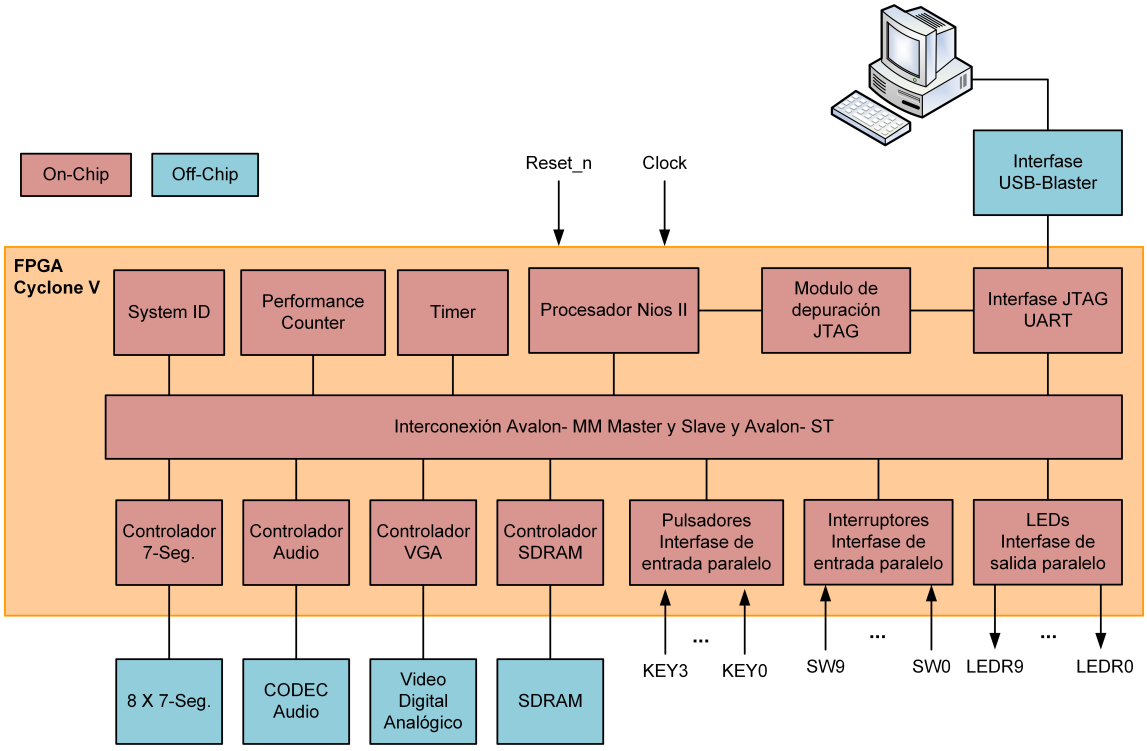

Figura 3. Sistema Integrado basado en Nios II.

Se utilizará la *Platform Designer* para realizar el diseño del hardware descrito en la *Figura 3*. A continuación, hay que realizar la asignación de pines con la Cyclone V para establecer las conexiones entre los pines del dispositivo y los componentes externos. Después se debe configurar la FPGA para implementar el sistema diseñado. Finalmente, se utilizará el entorno de desarrollo software Nios II SBT para compilar, descargar y ejecutar un programa de Nios, que permita verificar si el diseño realizado funciona correctamente. A lo largo de este ejercicio el alumno desarrollará los siguientes objetivos:

- Herramienta de desarrollo Quartus Prime.
- Uso de la *Platform Designer* para el diseño de sistemas integrados basado en Nios II.
- Características de temporización que debe respetarse en la SDRAM de la placa DE1-SoC.
- Uso de PLL (Phase-Locked Loop) para control de la temporización.
- Integración del sistema Nios II diseñado en el proyecto de Quartus Prime.
- Implementación del sistema diseñado en la placa DE1-SoC.
- Ejecución de un programa de aplicación en el procesador Nios II.

4. El Reloj del Sistema

Todos los elementos síncronos que hay dentro de una FPGA; por ejemplo, los registros configurados para actuar como flip-flops en los bloques de lógica programable, necesitan una señal de reloj. Dicha señal de reloj normalmente se origina en el exterior, entra en la FPGA a través de una de las entradas especiales para el reloj y entonces es encamina a través del dispositivo conectándola a los registros adecuados.

Por otra parte, los sistemas basados en microprocesadores también necesitan de una señal de reloj, genere las señales de sincronismo adecuadas y que determine el tiempo ejecución del programa.

En ambos casos, en los sistemas basados en FPGAs y en los sistemas microprocesadores, la señal de reloj determina la velocidad de funcionamiento del sistema, la cual debe ajustarse tanto a las especificaciones de la aplicación como a las limitaciones del hardware utilizado.

En esta ocasión se pretende implementar un sistema microprocesador en una FPGA. Por tanto, surge la necesidad de disponer de varias señales de reloj que se adapten a los diferentes requerimientos de los diversos bloques que formarán nuestro sistema. La elección de una buena estructura de relojes será fundamental para el correcto funcionamiento del sistema.

A continuación, se analizará el caso particular del uso de una memoria externa SDRAM y posteriormente se verá un ejemplo de estructura de relojes.

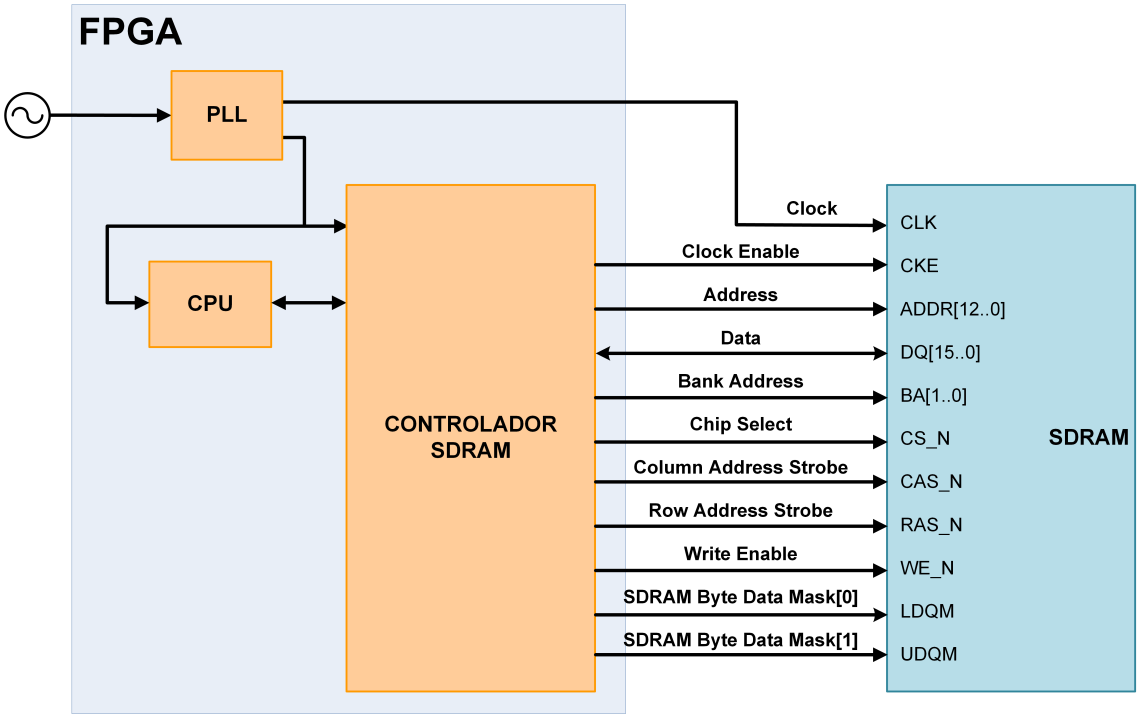

Figura 4. Señales de Conexionado de la SDRAM.

4.1. El Reloj y el Interfaz SDRAM

La placa DE1-SoC dispone de una memoria SDRAM con una capacidad de 64 Mbytes (32M x 16 bits). Las señales necesarias para comunicarse con las memorias SDRAM se muestran en la *Figura 4*. Todas las señales excepto la del reloj (Clock) son generadas por el controlador SDRAM que se puede crear utilizando la *Platform Designer*. La señal de reloj debe ser creada a parte, teniendo en cuenta los requerimientos temporales de *clock skew* que se comentarán más tarde. Cabe destacar que las señales activas a nivel bajo se denominan con el sufijo N.

El *clock skew* depende de las características físicas de la placa DE1-SoC. Para un correcto funcionamiento de la memoria SDRAM, es necesario que la señal de reloj de la memoria, se encuentre adelantada 3 ns respecto a la señal de reloj del sistema Nios II, para una frecuencia de reloj del sistema de 100 MHz. Esto se puede conseguir utilizando un PLL (*Phase-Locked Loop*).

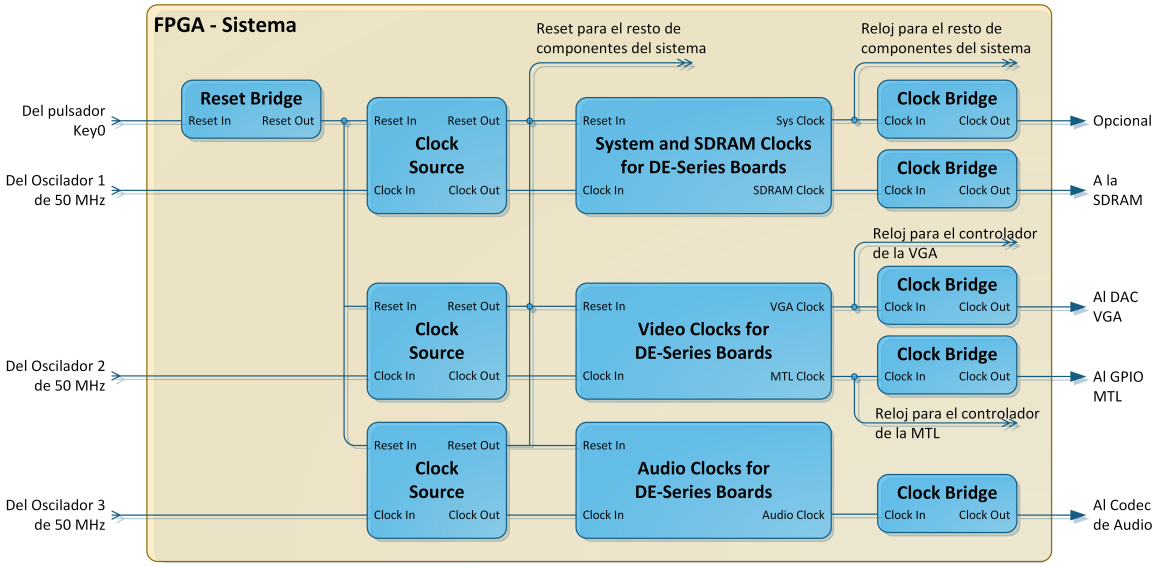

Figura 5. Estructura de relojes del sistema.

4.2. Propuesta de la Estructura de Relojes del Sistema

Tal como se acaba de comentar hace falta disponer de un reloj para la CPU que esté desfasado con respecto al reloj que se utilizará externamente para acceder a la memoria SDRAM, si a esto añadimos que el sistema puede necesitar de otros relojes para utilizar otros elementos externos como el controlador de la VGA o el Codec de Audio, tendremos que el número de relojes que hay que generar asciende a cinco: El reloj del sistema a 100MHz, el reloj para la memoria SDRAM de 100MHz con un desfase de 3 ns, el reloj de la VGA de 25 MHz, el reloj del códec de audio de 12,88 MHz y el reloj para una pantalla táctil o MTL de 33 MHz.

La creación de estos relojes se puede realizar de manera sencilla mediante el uso de uno o varios PLL, en función de las necesidades. Además de los PLLs, el sistema requiere del uso de otros elementos como el "*Clock Source*" para la entrada de las señales de reloj y los "*Clock Bridge*" para cuando se tienen que sacar las señales de reloj al exterior de la FPGA.

En la *Figura 5*, se muestra una propuesta de implementación de la estructura de relojes y del reset. En esta ocasión los PLLs se implementan mediante tres IPs del *University Program*: una para la generación del reloj del sistema y de la SDRAM, otra para la generación de los relojes de video y una tercera para la generación del reloj de audio. Tal como se ve, hace falta utilizar tres señales de reloj externas de 50 Mhz, para atacar a cada una de las IPs o PLLs, las cuales estan disponibles en la placa.

5. Cache de Datos en Nios II

En la versión "*Fast*" del procesador Nios II, se dispone de todas las prestaciones que el microprocesador de Altera, puede ofrecer. Esta versión es la única que puede utilizar cache de datos, además de la cache para instrucciones o de programa. Inicialmente cabe pensar que disponer de una cache para los datos, es una prestación que mejorará los tiempos de procesamiento en el caso de tener que realizar muchas operaciones de lectura y escritura de

datos en memoria. Sin embargo, lo que por defecto presuponemos que es una mejora, en el caso de uso de periféricos, puede ser contraproducente.

Cuando el procesador Nios II dispone de cache de datos, los datos se almacenan en la cache, en función del uso de los datos y de la ocupación de la cache, los datos se trasfieren a la memoria. En el caso particular de los periféricos, por ejemplo, un display 7 segmentos, al intentar escribir un dato en ellos para visualizarlo, este se almacena en la cache de datos, con lo cual no llega al periférico y por lo tanto no se llega a mostrar en los visualizadores. La solución que propone Altera, en el caso de los periféricos, consiste en eludir el uso de la cache de datos y escribir o leer directamente del periférico. Para ello, propone diferentes alternativas, que permiten saltarse el uso de la cache de datos en operaciones de lectura y escritura sobre periféricos.

- **El uso de macros de acceso directo a los periféricos** se trata de instrucciones concretas que permiten que el procesador Nios II se salte la cache de datos y acceda directamente a los periféricos. Estas instrucciones son:
    - IORD(base, offset), IOWR(base, offset, dato),
    - IORD_8DIRECT(base, offset), IOWR_8DIRECT(base, offset, dato),
    - IORD_16DIRECT(base, offset), IOWR_16DIRECT(base, offset, dato),
    - IORD_32DIRECT(base, offset), IOWR_32DIRECT(base, offset, dato).
- **El uso de funciones del HAL (Hardware Abstraction Layer)**, los periféricos suelen incorporar una librería de funciones para su uso, que permiten trabajar con un nivel de abstracción mayor, al emplear una capa de software denominada HAL. Estas funciones, permiten utilizar los periféricos sin problemas, aunque se disponga de cache de datos, ya que están preparadas para saltarse el uso de la cache, fundamentalmente porque internamente utilizan las macros comentadas en el punto anterior.
- **Mapa de memoria duplicado**, es uno de los mecanismos naturales que Nios II incorpora para eludir el uso de la cache de datos en los periféricos. El Nios II, es un procesador con un direccionamiento de 32 bits, esto permite tener un espacio de direccionamiento de 4 Gbytes. Sin embargo, esto no es correcto del todo, ya que los periféricos solo se pueden emplazar en los dos primeros Gbytes del mapa de memoria. Esto es debido a que el procesador Nios II, utiliza el bit de mayor peso del direccionamiento para eludir la cache de datos.

  Aquí puede surgir la duda, de realmente como está estructurado el mapa de memoria del procesador. La respuesta es sencilla, ver *Figura 6*, físicamente en la *Platform Designer* los periféricos se emplazan en los dos primeros Gbytes del mapa de memoria, pero virtualmente se replica el mismo mapa de memoria en los dos Gbytes más altos del mapa de memoria, de tal forma que a nivel de programación cada periférico dispone de dos direcciones base para acceder a él: la primera ubicada en los dos primeros Gbytes del mapa de memoria utilizará la cache de datos, mientras que la segunda estará ubicada en los dos Gbytes más altos del mapa de memoria y accederá al periférico sin hacer uso de la cache de datos, para ello simplemente habrá que coger la primera dirección base del periférico y activar el bit de mayor peso.

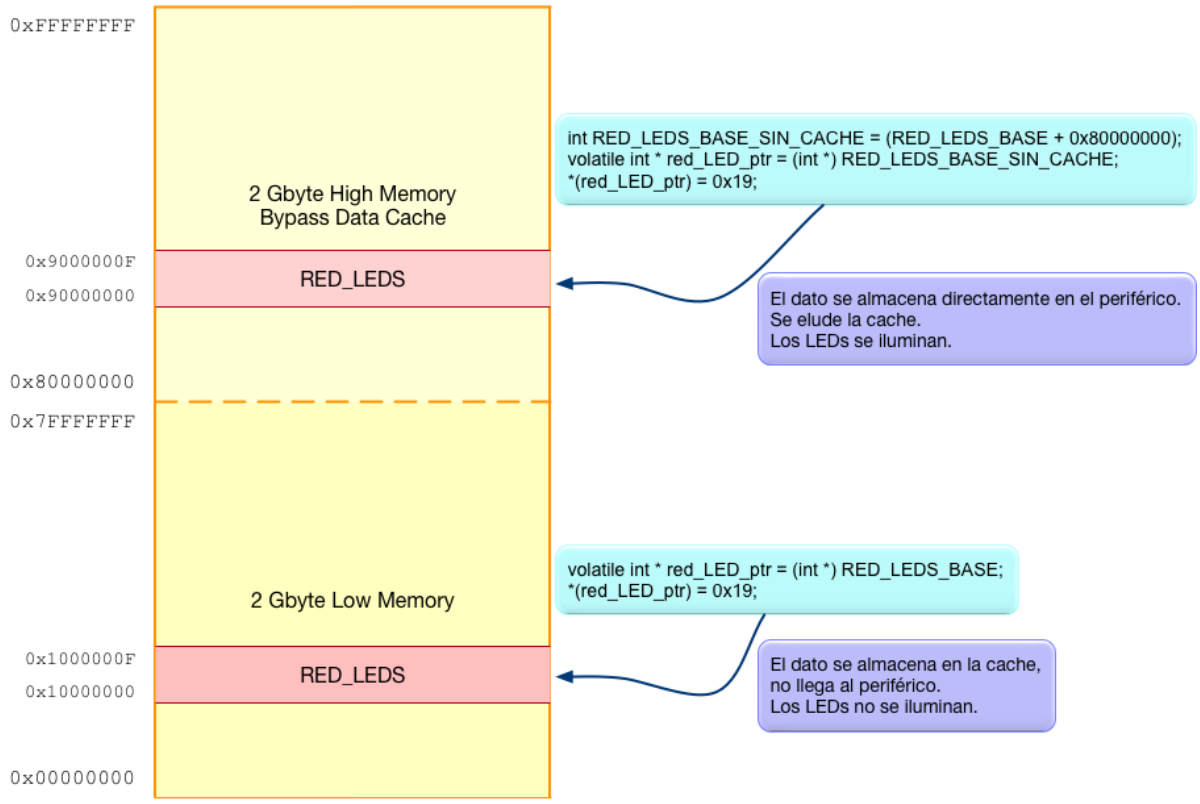

Figura 6. Duplicidad del mapa de memoria y bypass de cache de datos.

En la *Figura 6*, se puede ver que el puerto paralelo de los RED_LEDS, se encuentra ubicado en los 2 Gbytes más bajos de memoria en la dirección `0x10000000`. Si se accede a él, directamente mediante esta dirección, el dato se almacenará en la cache de datos y los Leds no se iluminarán. Sin embargo, el puerto de los RED_LEDS se encuentra replicado, virtualmente, en la parte alta de la memoria. Accediendo a esta dirección (`0x90000000`), se elude el uso de la cache de datos y los leds se iluminan. En la *Figura 6* se puede ver el fragmento de código de programación, que permite acceder al puerto de los RED_LEDS, eludiendo la cache de datos, simplemente activando el bit de mayor peso de la dirección base.

En la nueva generación del procesador Nios II, la duplicidad del mapa de memoria y el uso del bit de mayor peso del direccionamiento para eludir el uso de la cache de datos, en el acceso a los periféricos, es configurable cuando se instancia el procesador en la *Platform Designer*. En la *Figura 7* se puede ver la pestaña de "*Caches and Memory Interfaces*" del procesador, donde se configura, entre otras cosas, la cache de datos que se desea utilizar. En el apartado de la configuración de la cache de datos (*Data Cache*), se puede marcar la opción "*Use most-significant address bit in processor to bypass data cache"*, que permitirá disponer de un mapa de memoria duplicado con el funcionamiento que se ha comentado o por el contrario disponer de un mapa de memoria unificado de 4 Gbytes, sin posibilidad de eludir la cache de datos. Si se opta por desmarcar esta opción, los periféricos pueden eludir el uso de la cache de datos mediante la definición de una región especial, como se explica en el siguiente punto.

- **Definir una región de periféricos en el mapa de memoria**. En la nueva generación del procesador Nios II, se puede definir en el mapa de memoria una región para emplazar los periféricos que no se quiera que el procesador utilice la cache de datos para acceder a ellos. De esta manera, todos los periféricos que se encuentren dentro de esa región, serán accesibles directamente, eludiendo la cache por hardware, sin necesidad de recurrir al empleo de macros de programación específicas. Esta región se define en las opciones del procesador Nios II, cuando se emplaza en la *Platform*

Designer. Concretamente en la pestaña "*Caches and Memory Interfaces*" de los parámetros del procesador Nios II, que se muestra en la *Figura 7*. En dicha figura, se puede ver en la parte inferior, que se ha definido una región para periféricos (*Peripheral Region*) de 16 kbytes, a partir de la dirección de memoria `0x10000000`. Por tanto, se dispone a partir de dicha dirección de un espacio de 16 kbytes, para ubicar periféricos a los que se accederá directamente sin pasar por la cache de datos.

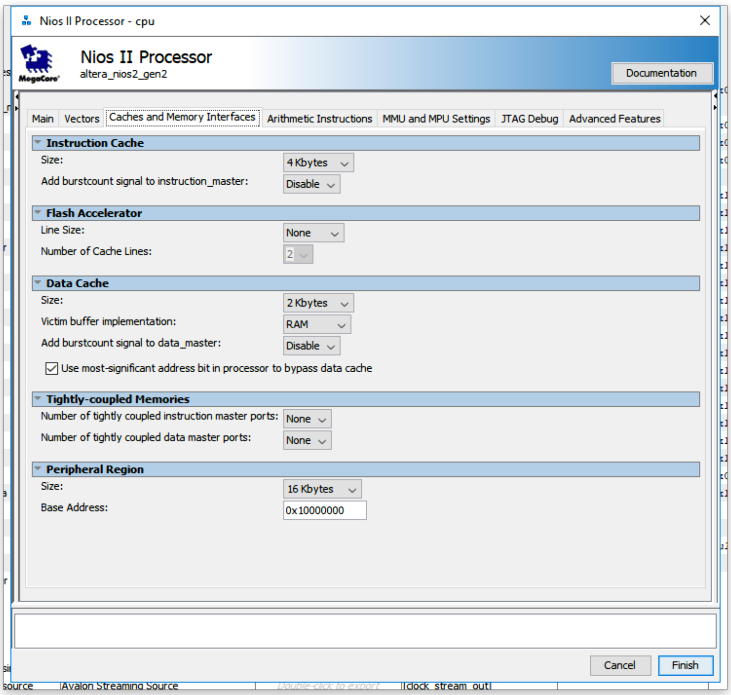

Figura 7. Selección Cache y memoria de periféricos del procesador Nios II.

6. Entorno de Visualización

6.1. Utilizando un monitor VGA

La placa DE1-SoC dispone de un conector del tipo D-SUB de 15 pines para conectar un monitor VGA. Tal como se muestra en la *Figura 8*, al conector le llegan directamente desde la FGPA Cyclone V las señales de sincronismo horizontal y vertical, mientras que las señales de color (RGB), se suministran a través de un convertidor digital analógico de Analog Devices ADV7123. El ADV7123 es un convertidor digital analógico de vídeo de 10 bits, aunque el diseño realmente utilice únicamente 8 bits, que permite generar imágenes SXGA con una resolución de 1280 × 1024 pixeles a una frecuencia de 100MHz.

El sistema de visualización que se quiere diseñar dispondrá de dos fuentes internas: una de ellas contendrá la imagen de fondo o background y la otra los caracteres o texto a representar en la pantalla también denominado foreground. Ambas fuentes se unirán superponiendo los caracteres a la imagen de fondo, por medio de un mezclador, para generar la imagen que se visualizará en la pantalla VGA. Tanto, la imagen de fondo como los caracteres que se quiere visualizar serán almacenados en una memoria o buffer, por medio del procesador Nios II. En la *Figura 9*, se puede ver el diagrama de bloques del sistema de visualización que se quiere implementar.

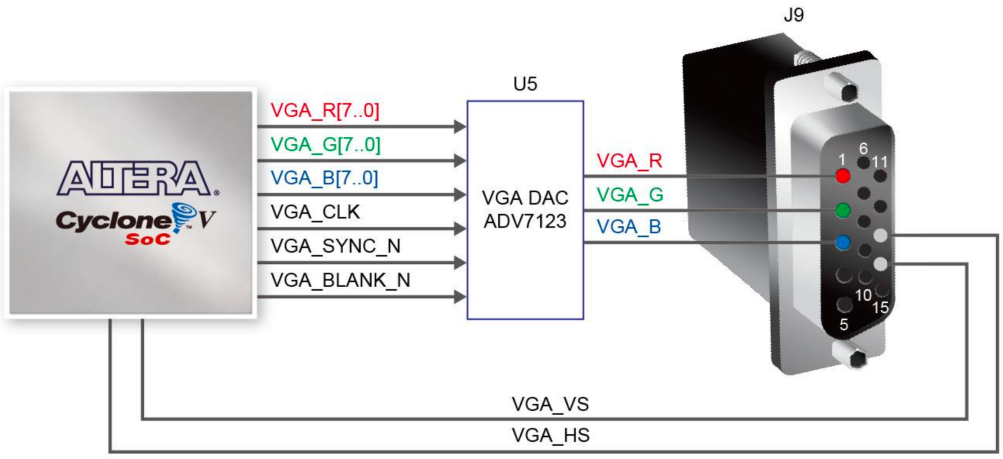

Figura 8. Esquema de conexionado entre la FPGA y el conector VGA en la placa DE1-SoC.

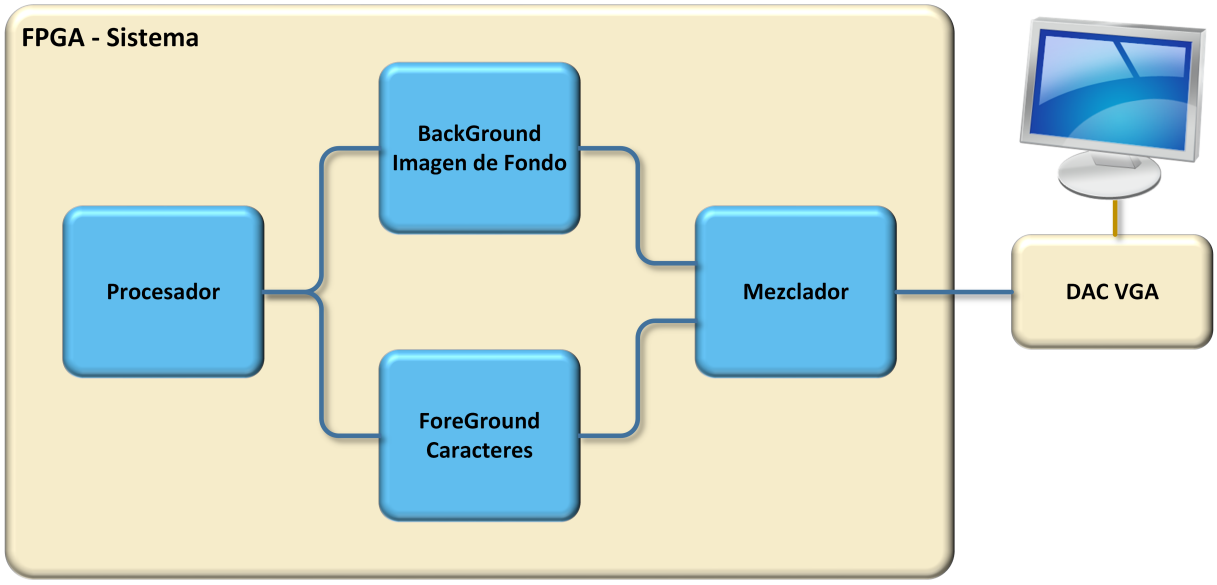

Figura 9. Planteamiento del sistema de visualización basado en Nios II para una pantalla VGA.

Para la implementación del entorno de visualización de la *Figura 9*, Altera ofrece un conjunto de bloques IP de Video, por medio de la biblioteca de componentes de la *Platform Designer*, que facilitan mucho la tarea de diseño del sistema. En realidad, Altera ofrece dos tipos diferenciados de bloques IP de Video denominados:

- *Video and Image Processing Suite*. Se trata de una "*Suite*" general de bloques IP para procesamiento de imagen y video, que requiere licencia. Se puede encontrar en la biblioteca de la *Platform Designer* en el apartado: **DSP > Video and Image Processing**.
- *Video IP Cores for Altera DE-Series Boards*. Se trata de un conjunto de bloques IP para procesamiento de video, que se encuentra enmarcado dentro del programa universitario de Altera y que es específico para ser utilizado en las placas de la serie DE. Esto implica que muchos parámetros ya se encuentran configurados para utilizar los recursos de la placa DE o que incluso el protocolo de transmisión de datos entre los diferentes bloques es propio. Estos bloques se pueden encontrar en la biblioteca de la *Platform Designer* en el apartado: **University Program > Audio & Video > Video**.

Con la finalidad de facilitar el diseño y dado que se va a ampliar un sistema que ya dispone de bloques IP de la biblioteca del programa universitario de Altera, se ha optado por utilizar únicamente bloques IP de video de programa universitario.

El programa universitario de Altera dispone de un total de 22 bloques IP de video, que se pueden utilizar para crear el entorno de visualización. Además, dispone de elementos adicionales de apoyo que también son necesarios como:

- *Video Clocks for DE-series Boards*, que por ejemplo generará los relojes necesarios para la VGA y para la pantalla MTL.
- *Audio and Video Config*, que permite configurar los elementos de video y audio de la placa DE.

Utilizando los bloques IP del programa universitario de Altera el sistema de visualización que hay que diseñar se muestra en la *Figura 10*. Tal como se muestra, los caracteres o foreground se implementan mediante un bloque IP denominado *Character Buffer for VGA Display*, el cual dispone de una memoria on-chip que es capaz de almacenar un total de 80 x 60 caracteres para la visualización VGA de la placa DE. Los caracteres se representarán mediante cuatro planos de 10 bits, de los cuales corresponden tres planos para el color (RGB) y un plano adicional para la transparencia de los caracteres. Estos se escribirán en el buffer desde el procesador Nios.

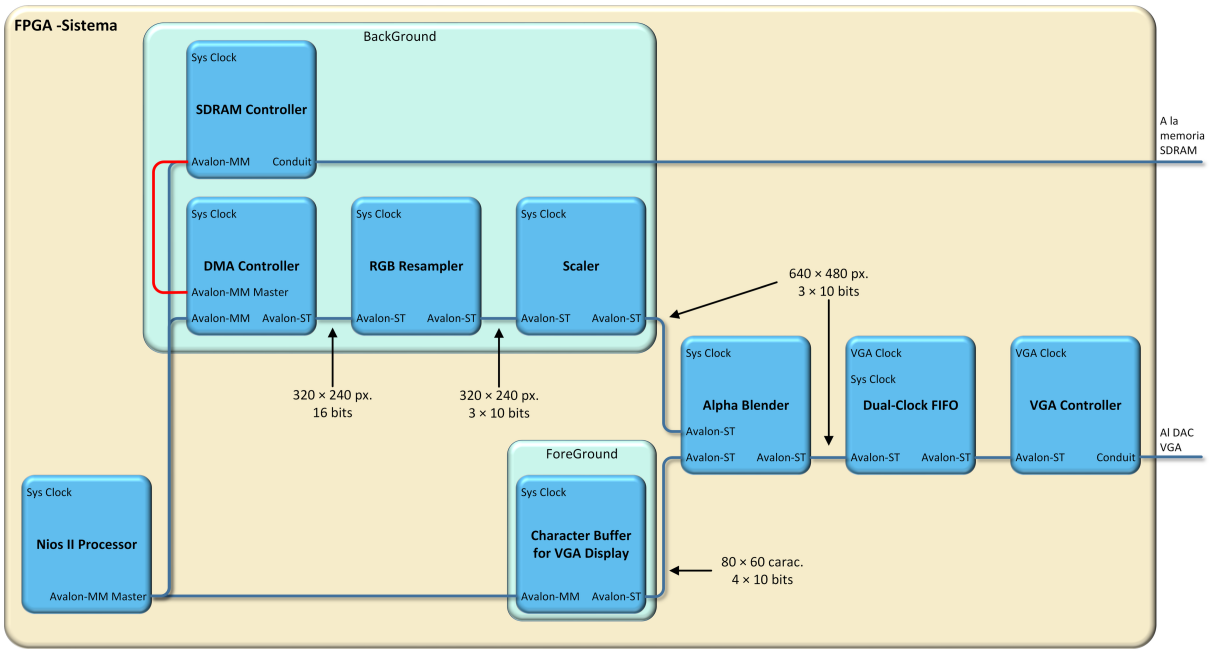

Figura 10. Esquema de los bloques a añadir en la Platform Designer para utilizar una pantalla VGA.

En cuanto a la imagen de fondo o background, será almacenada en la memoria SDRAM de la placa desde donde será leída mediante un controlador de DMA para visualizarla en la pantalla VGA. En este caso la memoria SDRAM actuará como memoria de programa y como memoria gráfica de la VGA. La imagen de fondo almacenada en la memoria SDRAM será de 320 × 240 pixeles y 16 bits para representar los tres colores RGB, por tanto, harán falta bloques IP adicionales que permitan convertir la imagen a un formato de 640 × 480 pixeles con tres planos de 10 bits para los colores RGB. Tal como se muestra en la *Figura 10*, la imagen de fondo necesita de los siguientes bloques IP:

- *Pixel Buffer DMA Controller.*
- *RGB Resampler.*
- *Scaler.*

Para concluir la visualización, tal como se muestra en la *Figura 10*, hacen falta los siguientes bloques IP:

- un *Alpha Blender*, que mezcle la imagen de fondo con los caracteres,
- un bloque *Dual-Clock FIFO*, encargado de sincronizar los datos entre dos relojes distintos, ya que el reloj del sistema es de 100 MHz, mientras que la VGA debe trabajar a 25 MHZ
- y por último el *VGA Controller* que genera las líneas necesarias con su temporización para actual sobre la DAC de la VGA.

Como ha nivel de hardware se va a utilizar la memoria SDRAM para la ejecución del programa y como memoria de video, es necesario que se definan dos zonas diferenciadas para que no hay solapes de uso de la memoria. Esta operación se hará desde la herramienta Nios II SBT (Software Build Tools), definiendo dos zonas de *linker* distintas dentro de la memoria, una como zona de programa y otra como memoria de video.

7. Pasos a seguir en la realización de un sistema utilizando la Platform Designer

La *Platform Designer* permite al diseñador crear un sistema basado en el procesador Nios II con una gran sencillez, simplemente seleccionando las unidades funcionales que se desea que tenga y especificando los parámetros o características de dichas unidades. Para implementar el sistema propuesto en la *Figura 3* se deben instanciar las siguientes unidades funcionales:

- El procesador Nios II, como una unidad central de proceso (CPU).
- El Reloj del Sistema, según la *Figura 5*.
  - PLL para la creación de los diferentes relojes.
  - Fuentes de Reloj.
  - Fuentes de Reset.
  - Puentes de Reloj.
- Controlador de memoria SDRAM de 64 MBytes.
- Cuatro interfaces paralelos de entrada salida para: los interruptores, los pulsadores, los leds rojos y los displays 7-segmentos.
- Una interfase JTAG UART para la comunicación con el PC.
- Un Timer.
- Un Performance Counter.
- Un componente de identificación del sistema.
- Una UART, para comunicaciones serie RS-232.
- Controlador de teclado y/o ratón PS2.
- Controlador del códec de Audio.
- Los IP de visualización en una VGA mostrados en la *Figura 10*.

Para definir y crear el sistema integrado propuesto, se comienza ejecutando el software de Altera Quartus Prime y a continuación se seguirán los siguientes pasos:

1 Crear un nuevo proyecto Quartus Prime para el sistema. Tal como se muestra en la *Figura 11*, se almacena el nuevo proyecto en un directorio denominado `DE1_SoC_Media_Computer`, y se le asigna el nombre de `DE1_SoC_Media_Computer`,

tanto al proyecto como a la entidad jerárquica superior del diseño. Cada uno puede escoger el directorio o el nombre de proyecto que desee. En este proyecto se escogerá el dispositivo 5CSEMA5F31C6 de la familia Cyclone V, ya que es la FPGA que tiene la placa DE1-SoC.

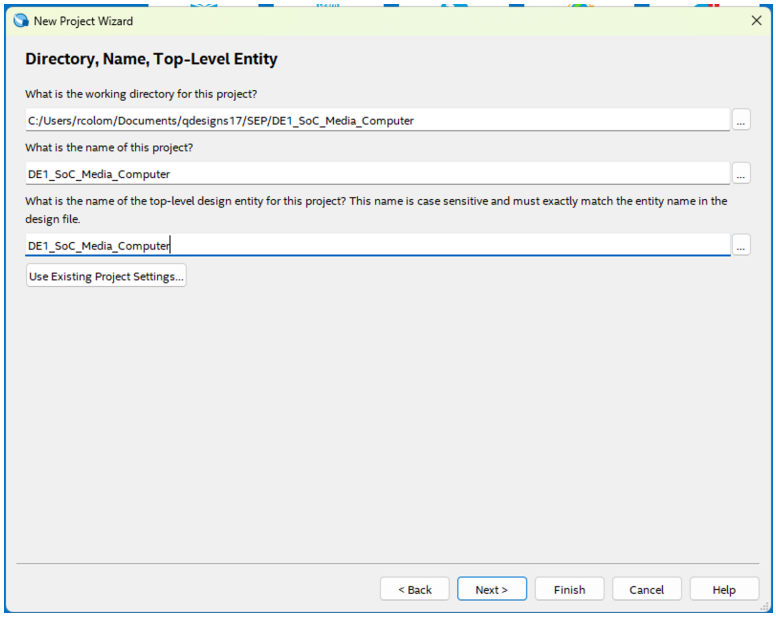

Figura 11. Crear un nuevo proyecto.

2 Seleccionar: **Tools > Platform Designer**, esto provocará que se abra la herramienta *Platform Designer* para la creación del sistema integrado, tal como se aprecia en la *Figura 12*.

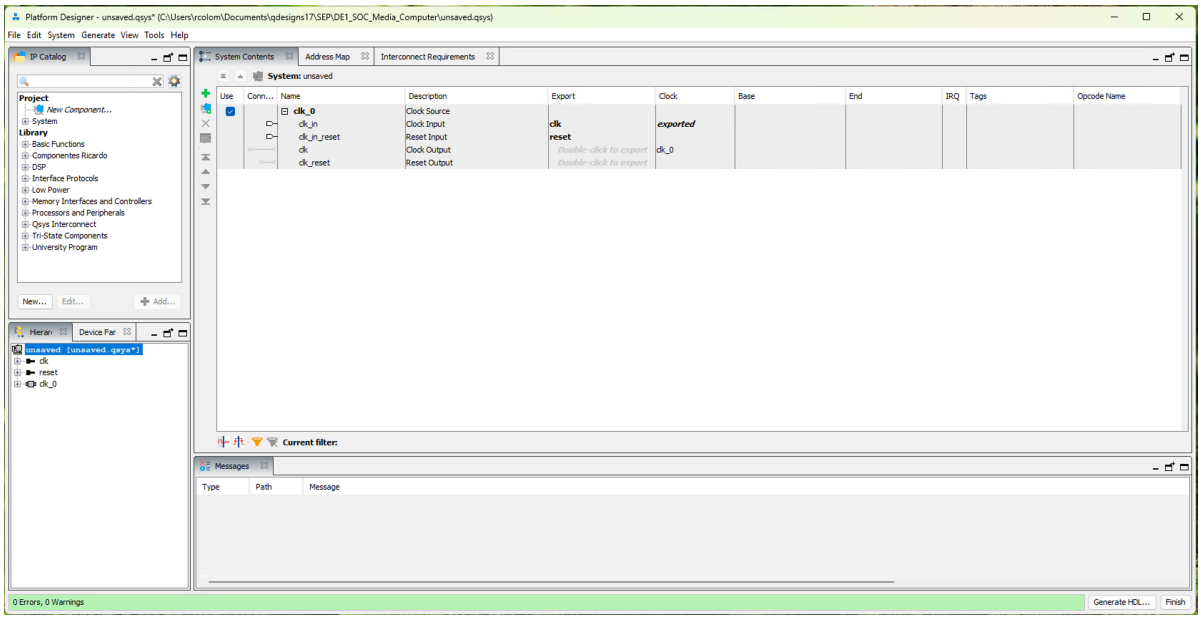

Figura 12. Crear un nuevo sistema Nios II.

3 La *Figura 12*, muestra la pestaña de contenidos del sistema de la *Platform Designer*, la cual se utiliza para añadir componentes al sistema y configurar los componentes que el diseño requiere. En el lado izquierdo de la ventana se muestra la lista de componentes disponibles, los cuales se encuentran clasificados por su funcionalidad. Cabe destacar que inicialmente,

ya aparece un componente en la pestaña de contenidos del sistema, es la fuente de reloj (*Clock Source*).

4 El procesador Nios II funciona bajo el control de una señal de reloj, proveniente de un oscilador externo. Altera exige que el oscilador externo pase a través del componente denominado "*Clock Source*" para posteriormente ser utilizado en el resto del sistema. En este ejercicio se implementará la estructura de relojes mostrada en la *Figura 5*. Para comenzar, se renombrará el "*Clock Source*" inicial:

- En pestaña de **System Contents** se selecciona el **Clock Source** y se renombra como *clk_50*.
- En la columna **Export** de la pestaña **System Contents**, aparecen los nombres que tendrán las señales de entrada y salida al sistema, en esta ocasión hay que cambiar, el nombre de la señal de reloj por: `clk_50_in`.

5 Como hacen falta tres relojes externos de 50 MHz, habrá que añadir dos "*Clock Source*" adicionales.

- Seleccionar: **Basic Funtions > Clocks; PLLs and Resets > Clock Source** y pulsar en **Add**, para lanzar la ventana de configuración del componente.
- Establecer una frecuencia conocida de 50 MHz.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como *clk_50_2*.
- Renombrar la señal de entrada del puerto `Clock Input` como `clk_50_2_in`.

6 Añadir un tercer "*Clock Source*" para otro reloj de 50 MHz.

- Seleccionar: **Basic Funtions > Clocks; PLLs and Resets > Clock Source** y pulsar en **Add**, para lanzar la ventana de configuración del componente.
- Establecer una frecuencia conocida de 50 MHz.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como *clk_50_3*.
- Renombrar la señal de entrada del puerto `Clock Input` como `clk_50_3_in`.

7 Añadir el "*Reset Bridge*" para la entrada de la señal de reset.

- Seleccionar: **Basic Funtions > Bridges and Adaptors > Reset > Reset Bridge** y pulsar en **Add**, para lanzar la ventana de configuración del componente.
- Seleccionar la opción **Active low reset**.
- Establecer, **Synchronous Edges: None**
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como *reset_bridge*.
- Renombrar la señal de entrada del puerto `in_reset` como `reset_bridge_in`.
- Conectar el puerto `out_reset` del *reset_bridge* con los puertos `clk_in_reset` del *clk_50*, del *clk_50_2* y del *clk_50_3*.

8 Tal como se ha comentado anteriormente hay que utilizar PLLs para la creación de los relojes internos. En este caso se utilizarán tres componentes que contienea los parámetros necesarios para crear los relojes de la DE1-SoC.

- Seleccionar: **University Program > Clock > System and SDRAM Clocks for DE-series Boards** y pulsar en **Add**, para lanzar la ventana de configuración del componente.
- En la ventana de configuración de este componente que se muestra en la *Figura 13*, seleccionar la placa DE1-SoC y establecer el reloj del sistema en 100 MHz.

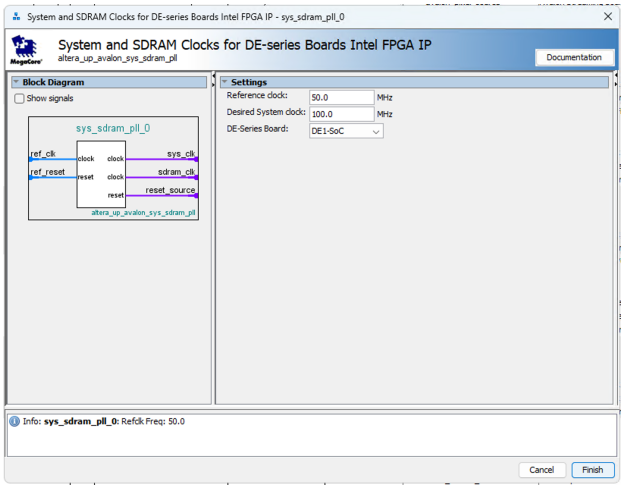

Figura 13. Configuración del System and SDRAM Clocks for DE-series Boards.

- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como *sys_sdram_pll*.
- Conectar el puerto `ref_clk` del *sys_sdram_pll* con el puerto `clk` del *clk_50*.
- Conectar el puerto `ref_reset` del *sys_sdram_pll* con el puerto `clk_reset` del *clk_50*.
- Seleccionar: **University Program > Clock > Video Clocks for DE-series Boards** y pulsar en **Add**, para lanzar la ventana de configuración del componente.
- En la ventana de configuración de este componente que se muestra en la *Figura 14*, seleccionar las opciones que se muestran en dicha figura.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como *video_pll*.
- Conectar el puerto `ref_clk` del *video_pll* con el puerto `clk` del *clk_50_2*.
- Conectar el puerto `ref_reset` del *video_pll* con el puerto `clk_reset` del *clk_50_2*.
- Seleccionar: **University Program > Clock > Audio Clocks for DE-series Boards** y pulsar en **Add**, para lanzar la ventana de configuración del componente.

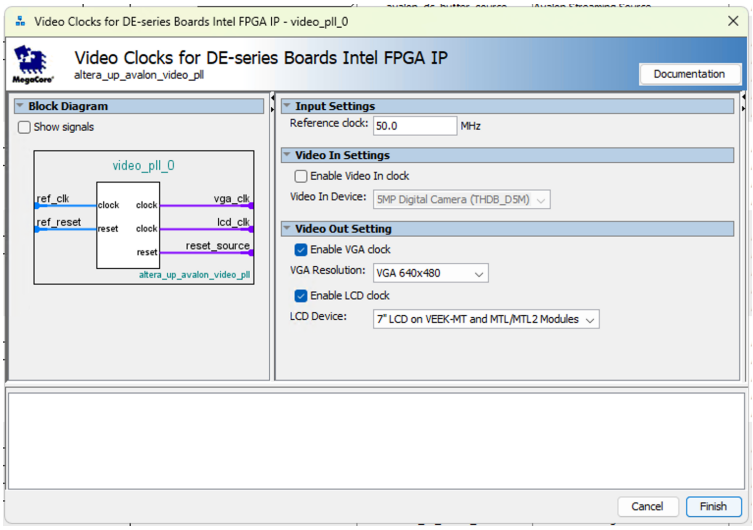

Figura 14. Configuración del Video Clocks for DE-series Boards.

- En la ventana de configuración de este componente que se muestra en la *Figura 15*, seleccionar las opciones que se muestran en dicha figura.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como *audio_pll*.
- Conectar el puerto `ref_clk` del *audio_pll* con el puerto `clk` del *clk_50_3*.
- Conectar el puerto `ref_reset` del *audio_pll* con el puerto `clk_reset` del *clk_50_3*.

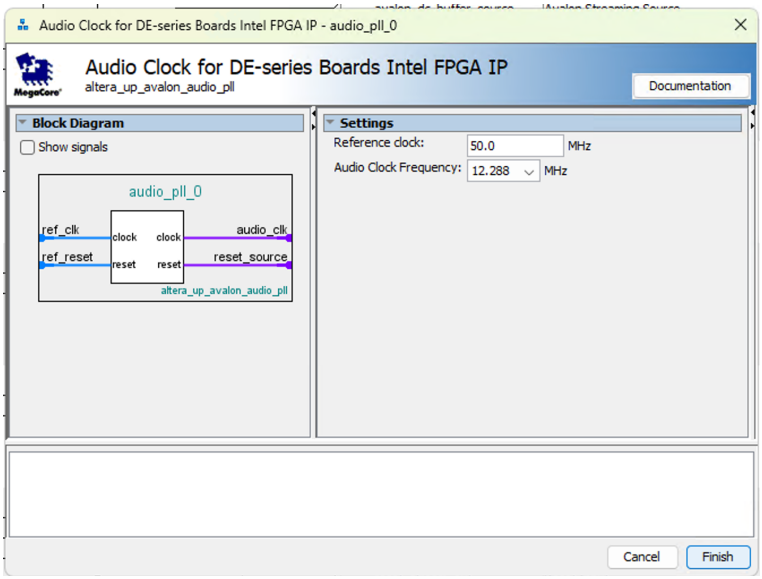

Figura 15. Configuración del Audio Clocks for DE-series Boards.

9 Para concluir con la generación de los relojes, hay que añadir los "*Clock Bridges*" que permitirán extraer las señales de cada uno de esos relojes.

- Seleccionar: **Basic Funtions > Bridges and Adaptors > Clock > Clock Bridge** y pulsar en **Add**, para lanzar la ventana de configuración del componente, que en este caso no que modificar ningún parámetro.

- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente Clock Bridge, como *sys_clk*.
- Conectar el puerto `in_clk` del *sys_clk* al puerto `sys_clk` del *sys_sdram_pll*.
- En la fila del puerto `out_clk` del *sys_clk*, hacer click en la columna **Export** para exportar la salida de reloj del controlador Clock Bridge, nombrarla como `sys_clk_out`.
- Repetir el proceso añadiendo cuatro Clock Bridge más, nombrándolos como *sdram_clk*, *vga_clk, mtl_clk* y *audio_clk*.
- Conectar el puerto `in_clk` del *sdram_clk* al puerto `sdram_clk` del *sys_sdram_pll*.
- Conectar el puerto `in_clk` del *vga_clk* al puerto `vga_clk` del *video_pll*.
- Conectar el puerto `in_clk` del *mtl_clk* al puerto `lcd_clk` del *video_pll*.
- Conectar el puerto `in_clk` del *audio_clk* al puerto `audio_clk` del *audio_pll*.
- Exportar el puerto `out_clk` del *sdram_clk*, como `sdram_clk_out`.
- Exportar el puerto `out_clk` del *vga_clk*, como `vga_clk_out`.
- Exportar el puerto `out_clk` del *mtl_clk*, como `mtl_clk_out`.
- Exportar el puerto `out_clk` del *audio_clk*, como `audio_clk_out`.

10 En este paso se añadirá al sistema su componente fundamental: el procesador Nios II. Para ello se realizará lo siguiente:

- Seleccionar: **Processors and Peripherals > Embedded Processors > Nios II Processor** y pulsar en **Add**, para lanzar la ventana de configuración del componente.
- En la pestaña **Main**, en la opción **Nios II Core**, se selecciona la versión del procesador rápida, **Nios II/f**, como se muestra en la *Figura 16*.
- En la pestaña de **Caches and Memory Interfaces**, se dejan los valores por defecto para la **Instruction Cache** y la **Data Cache**. Mientras que en **Peripheral Region**, se selecciona un tamaño de 16 kbytes y se establece la dirección base en `0x10000000`, como se muestra en la *Figura 18*.
- En la pestaña de **JTAG Debug**, se selecciona el Level 2, mediante las opciones que se muestran en la *Figura 19*. Se pulsa directamente en el botón de **Finish** para volver a la ventana principal de la *Platform Designer*.
- Renombrar el componente Nios II Processor, como *cpu*.
- Conectar el puerto `clk` del *cpu* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` de la *cpu* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.

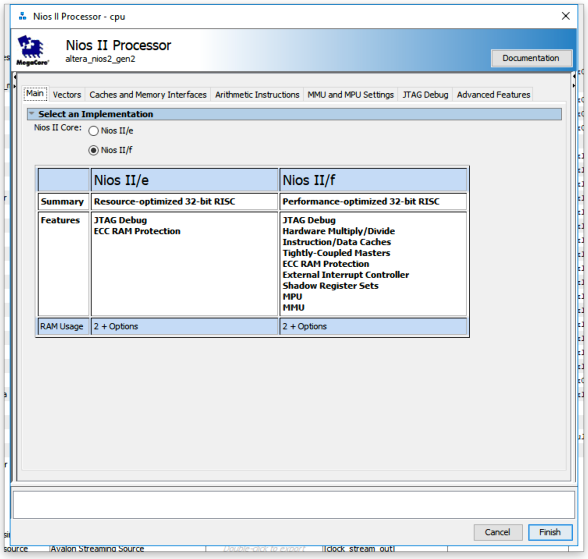

Figura 16. Selección Core del procesador Nios II.

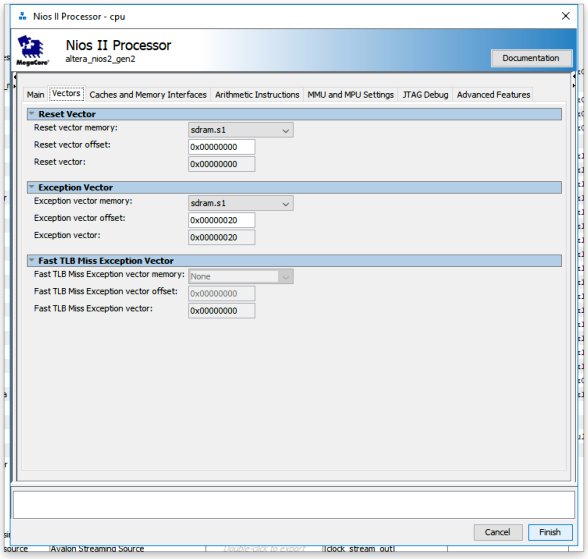

Figura 17. Establecimiento del vector de reset y de excepción del procesador Nios II.

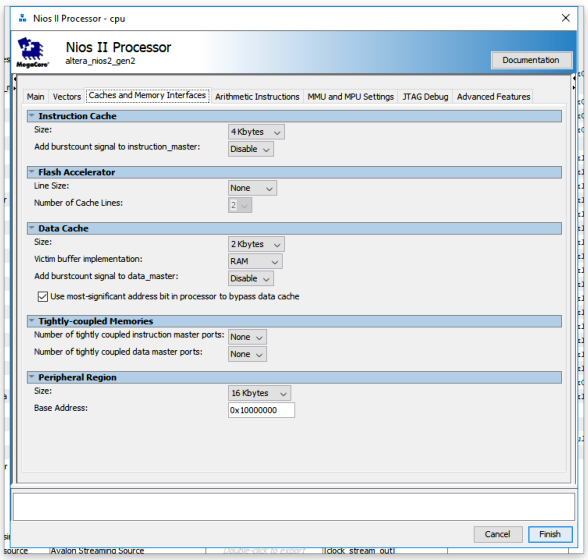

Figura 18. Selección Cache y memoria de periféricos del procesador Nios II.

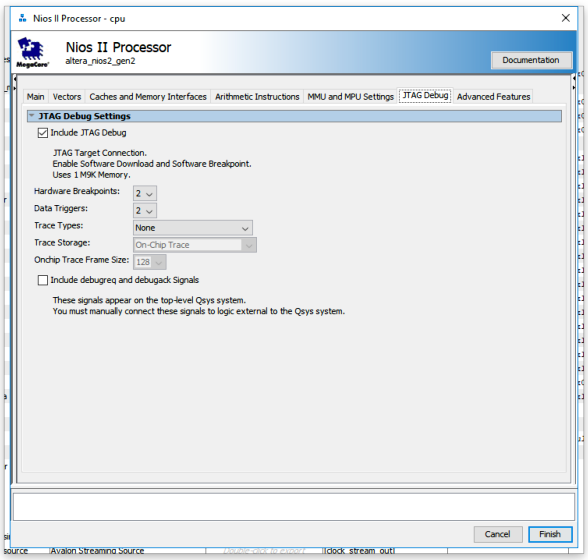

Figura 19. Selección nivel de depuración del procesador Nios II.

11 El siguiente componente para añadir es el controlador de SDRAM, para el cual se procederá como sigue:

- Seleccionar: **Memory Interfaces and Controllers > SDRAM > SDRAM Controller** y pulsar en **Add**.
- En la ventana de configuración del controlador SDRAM, que se muestra en la *Figura 20*, se debe establecer un tamaño de palabra de 16 bits, con una arquitectura de 4 bancos en un solo componente, distribuidos en 13 filas y 10 columnas, lo que hace un total de 64 MBytes o 512 Mbits.

- Seleccionar la opción de **Include a funcional memory model in the system testbech**.
- Los parámetros de temporización de la pestaña de **Timing** se deben establecer tal como muestra la *Figura 20*.
- Pulsar el botón de **Finish** y la memoria aparecerá en ventana de contenidos del sistema.
- Renombrar el componente SDRAM, como *sdram*.
- Conectar el puerto `clk` de la *sdram* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` de la *sdram* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `s1` de la *sdram* al puerto `data_master` y al `instrucción_master` de la *cpu*.
- Fijar la dirección de memoria base en el valor `0x0000_0000`.
- En la fila `wire` de la *sdram*, hacer click en la columna **Export** para exportar los puertos del controlador para conectar la SDRAM.

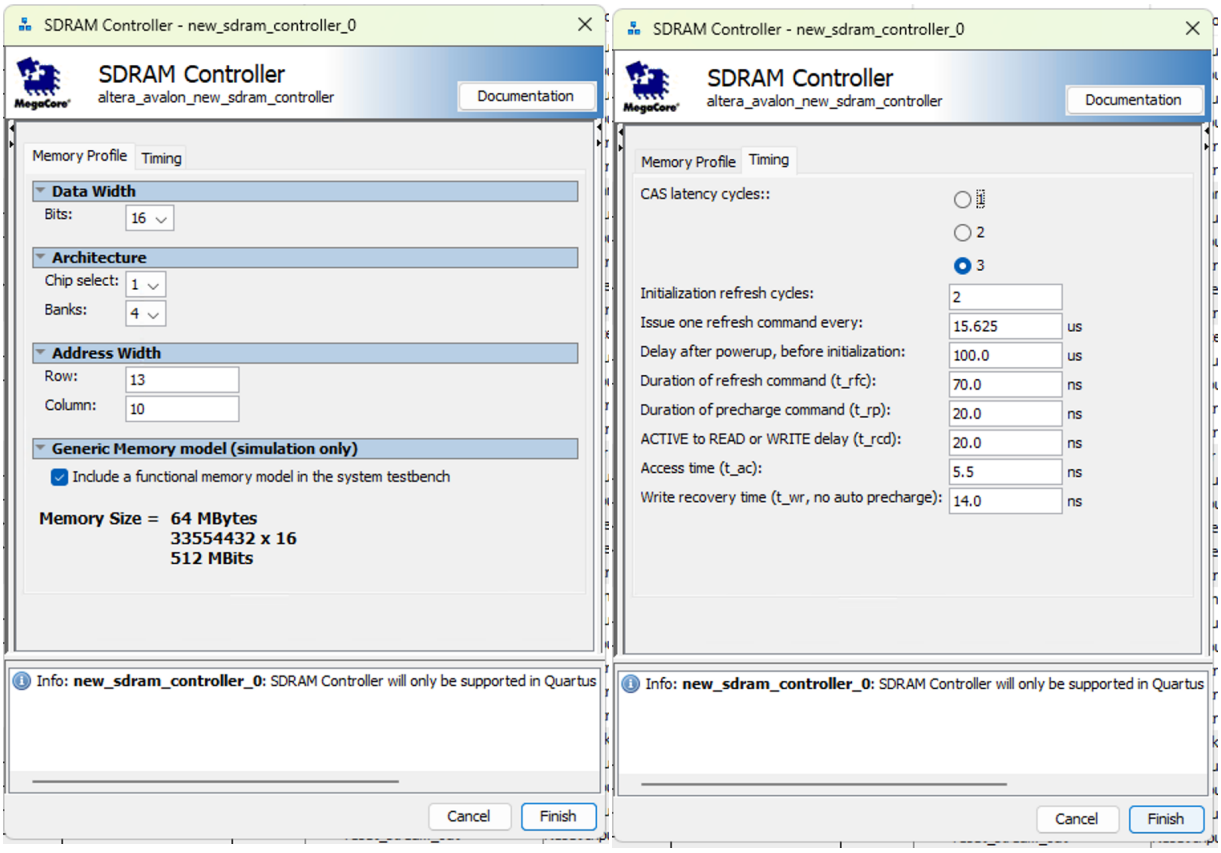

Figura 20. Definición del controlador SDRAM.

12 En algunos momentos nos puede hacer falta una memoria On-Chip, ya sea RAM o ROM. En este punto se va a añadir esa memoria.

- Seleccionar: **Basic Functions > On-Chip Memory > On-Chip Memory (RAM or ROM)** y pulsar en **Add**.
- En la ventana de configuración de la memoria On-Chip, que se muestra en la *Figura 21*, se establece un tamaño de palabra de 16 bits con una memoria total de 262144 bytes.
- Además, hay que marcar la opción *Enable In-System Memory Content Editor feature*, estableciendo como *Instance ID* el valor **IMGN**. El resto de los parámetros quedaran igual.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar la memoria como *onchip_memory.*
- Conectar el puerto `clk1` de la *onchip_memory* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset1` de la *onchip_memory* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `s1` de la *onchip_memory* al puerto `data_master` de la *cpu*.
- Fijar la dirección de memoria base en el valor `0x0800_0000`.

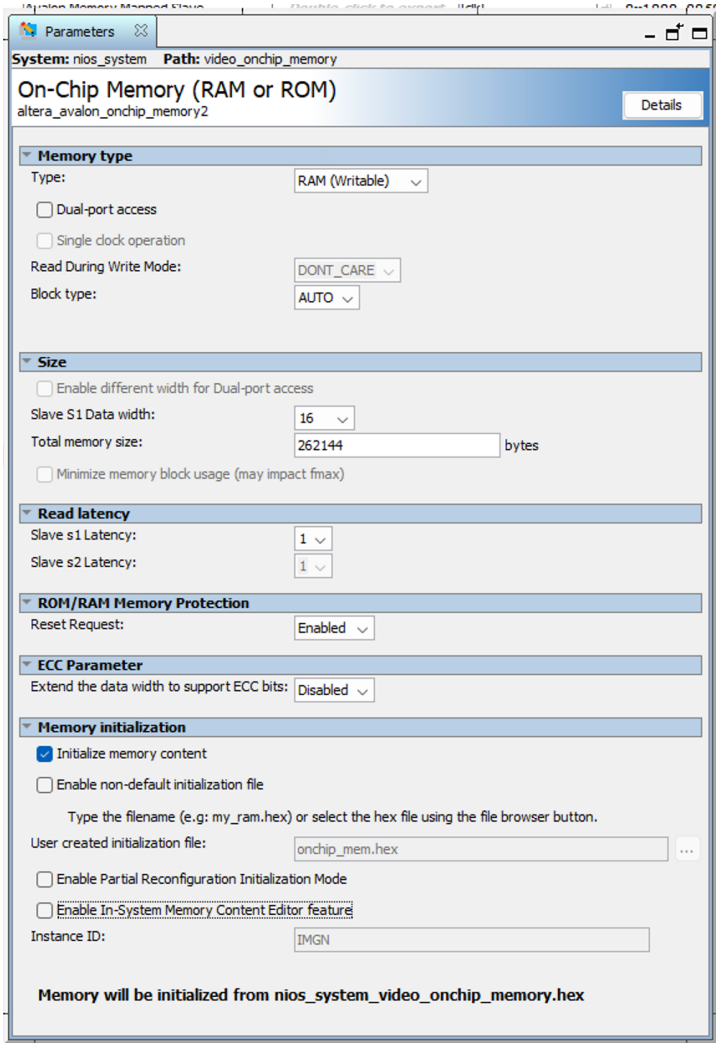

Figura 21. Definición de la memoria On-Chip.

13 Una vez se han especificado las memorias del sistema, se puede terminar la configuración del procesador Nios II, escogiendo cual será la memoria que contendrá el Vector de Reset y los Vectores de Excepción. El procedimiento para modificar los aspectos del procesador Nios II comentados será el siguiente:

- En pestaña de **System Contents** se pincha dos veces sobre el procesador Nios II (la *cpu*), con lo cual se despliega la ventana de la *Figura 16*.
- En la pestaña **Vectors**, el apartado de **Reset Vector**, se selecciona el `sdram.s1` para el **Reset vector memory** y `0x00000000` para el **Reset vector offset**.
- En el apartado de **Exception Vector**, se selecciona el `sdram.s1` para el **Exception vector memory** y `0x00000020` para el **Exception vector offset**. Tal como se ve en la *Figura 17*.
- Conectar el puerto `debug_mem_slave` de la *cpu* al puerto `data_master` y al `instruction_master` de la *cpu*.
- Fijar la dirección de memoria base del `debug_mem_slave` de la *cpu*, en el valor `0x0400_8000`.

14 Es necesario realizar una conexión entre el PC y la placa DE1-SoC para establecer un mecanismo de comunicación entre el sistema Nios II y el PC que permita el control de este. Esto es posible mediante la instanciación de una interfaz JTAG UART, de la siguiente manera:

- Seleccionar: **Interface Protocols > Serial > JTAG UART** y pulsar en **Add**, para lanzar la ventana de configuración mostrada en la *Figura 22*.
- No cambiar ningún parámetro de los que se establecen por defecto.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como: *jtag_uart*.
- Conectar el puerto `clk` del *jtag_uart* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *jtag_uart* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_jtag_slave` del *jtag_uart* al puerto `data_master` de la *cpu*.
- Conectar el puerto `irq` del *jtag_uart* al puerto `irq` de la *cpu*.
- Fijar la dirección de memoria base en el valor `0x1000_0000`.
- Fijar el nivel de interrupción, en la columna IRQ en el valor `8`.

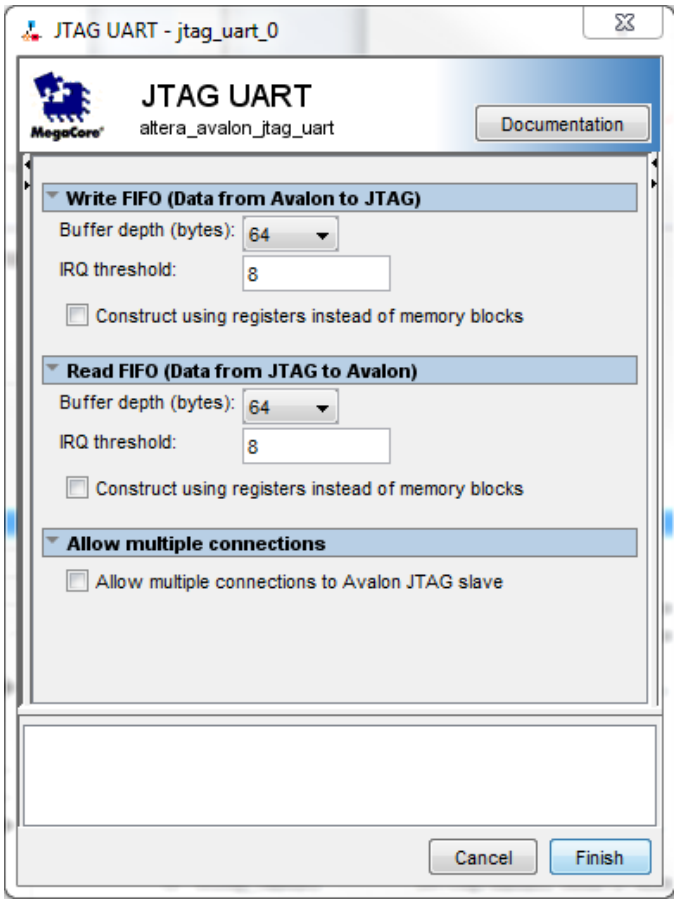

Figura 22. Definición de la interfase JTAG UART.

15 En muchos sistemas es necesario un componente que permita realizar cálculos temporales con precisión. En este caso Nios II dispone en su librería de un **Interval Timer**.

- Seleccionar: **Processors and Peripherals > Peripherals > Interval Timer** y pulsar en **Add**, para lanzar la ventana de configuración mostrada en la *Figura 23*.
- En el apartado de la derecha denominado **Presets** seleccionar la opción: **Full-featured** y pulsar en **Apply**.
- Establecer el **Period** en **1 ms**. El resto de los parámetros dejar sus valores por defecto tal como aparecen en la *Figura 23*.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como: *timer*.
- Conectar el puerto `clk` del *timer* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *timer* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `s1` del *timer* al puerto `data_master` de la *cpu*.
- Conectar el puerto `irq` del *timer* al puerto `irq` de la *cpu*.
- Fijar la dirección de memoria base en el valor `0x1000_0020`.
- Fijar el nivel de interrupción, en la columna IRQ en el valor `0`.

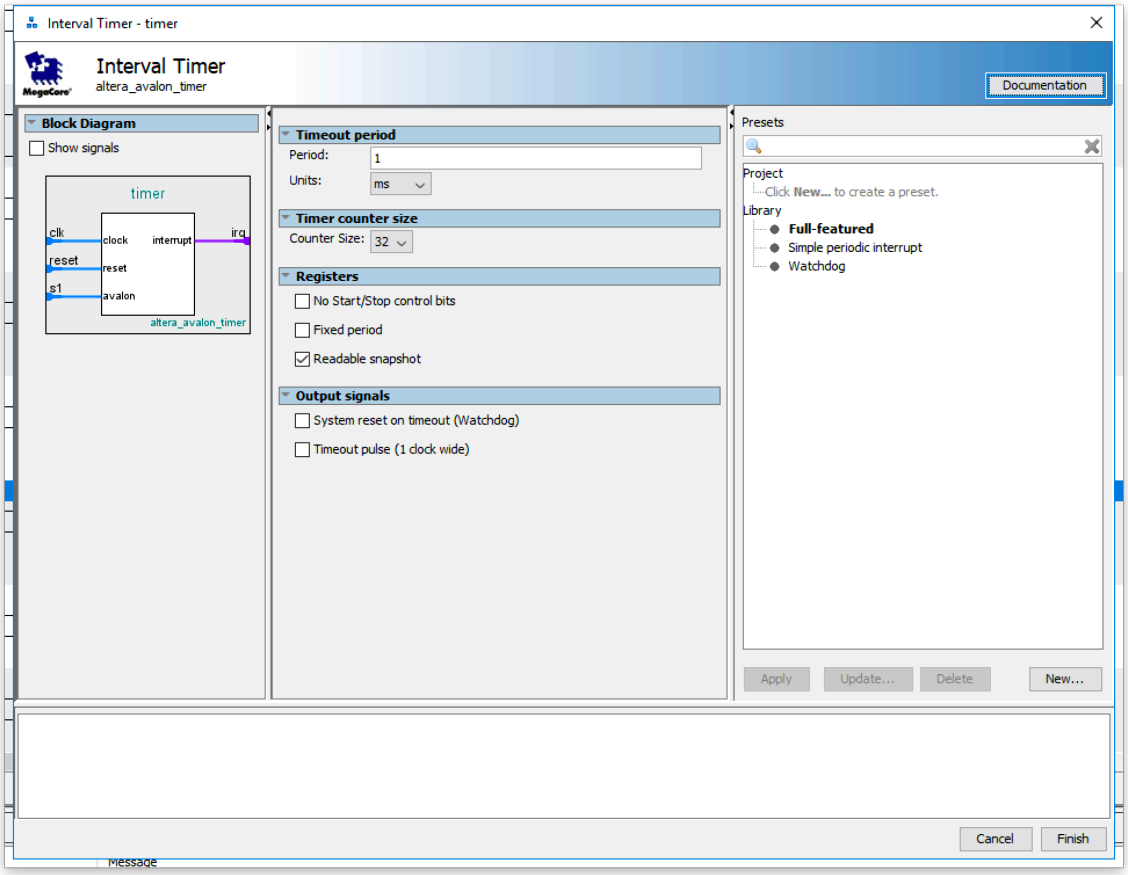

Figura 23. Definición del Interval Timer.

16 Ahora se añadirá un componente (**System ID Peripheral**) que se utiliza para identificar el sistema Nios II. Su principal función consiste en evitar que se pueda cargar un programa software que haya sido creado para otro sistema Nios II distinto.

- Seleccionar: **Basic Functions > Simulation; Debug and Verification > Debug and Performance > System ID Peripheral** y pulsar en **Add**, para lanzar la ventana de configuración mostrada en la *Figura 24*.
- No cambiar ningún parámetro de los que se establecen por defecto.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como: *sysid*.
- Conectar el puerto `clk` del *sysid* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *sysid* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `control_slave` del *sysid* al puerto `data_master` de la *cpu*.
- Fijar la dirección de memoria base en el valor `0x1000_0010`.

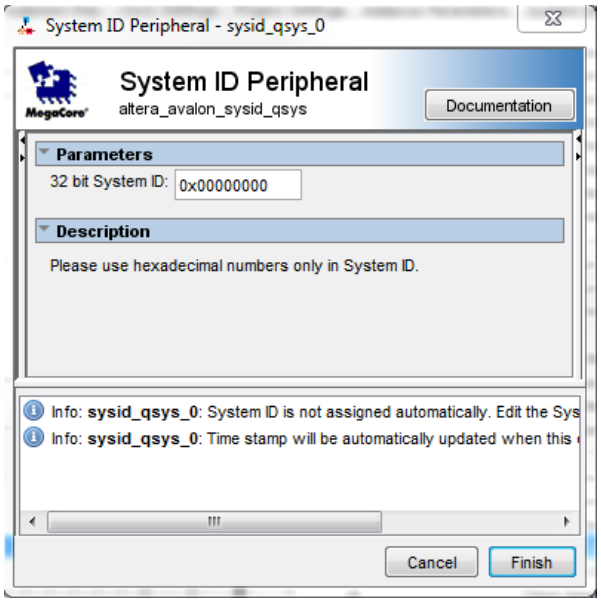

Figura 24. Definición del System ID Peripheral.

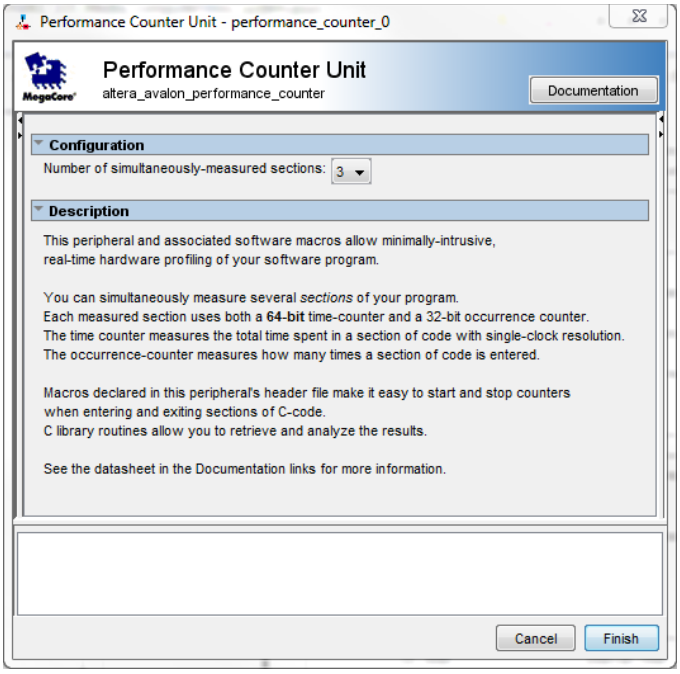

Figura 25. Definiendo las características del contador.

17 Se añade un contador para evaluar el rendimiento del sistema del siguiente modo:

- Seleccionar: **Basic Functions > Simulation; Debug and Verification > Debug and Performance > Performance Counter Unit** y pulsar en **Add**, para lanzar la ventana de configuración del temporizador mostrada en la *Figura 25*, en la que se dejan las opciones por defecto.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente Performance Counter Unit, como *performance_counter*.

- Conectar el puerto `clk` del *performance_counter* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *performance_counter* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `control_slave` del *performance_counter* al puerto `data_master` de la *cpu*.
- Fijar la dirección de memoria base en el valor `0x1000_0040`.

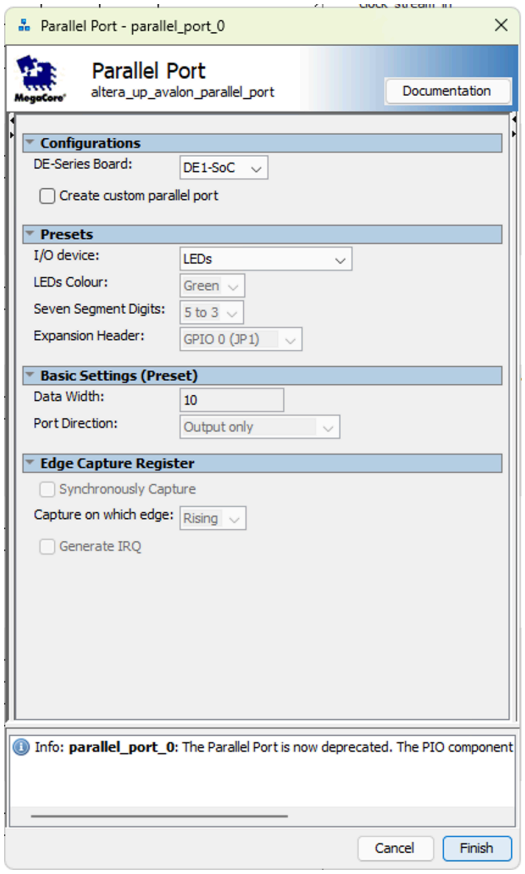

Figura 26. Definiendo las características del puerto para los LEDs rojos.

18 Establecer los puertos paralelos de entrada y salida para los diferentes elementos de la placa DE1-SoC. En primer lugar, los LEDs rojos:

- Seleccionar: **University Program > Generic IO > Parallel Port** y pulsar en **Add**, para lanzar la ventana de configuración mostrada en la *Figura 26*.
- Establecer los parámetros tal como aparecen en la *Figura 26*.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como: *red_leds*.
- Conectar el puerto `clk` de los *red_leds* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` de los *red_leds* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.

- Conectar el puerto `avalon_parallel_port_slave` de los *red_leds* al puerto `data_master` de la *cpu*.
- Fijar la dirección de memoria base en el valor `0x1000_0090`.
- En la fila `external_interface` de los *red_leds*, hacer click en la columna **Export** para exportar los puertos.

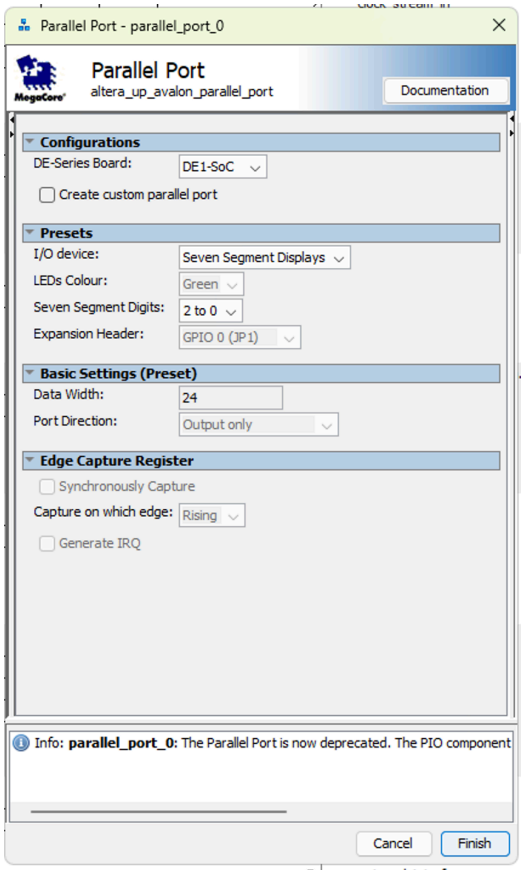

Figura 27. Definiendo las características del puerto para los display 7-Segmentos HEX0 a HEX2.

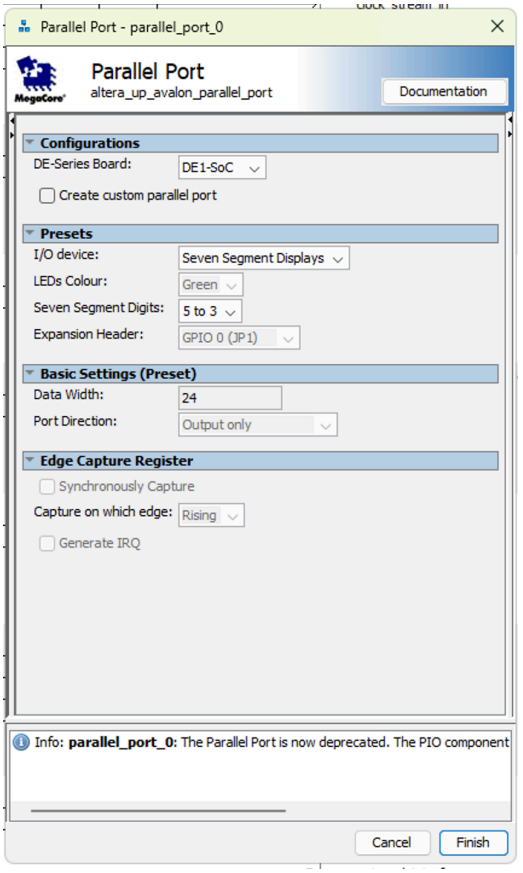

Figura 28. Definiendo las características del puerto para los display 7-Segmentos HEX3 a HEX5.

19 Establecer el puerto paralelo para los displays 7-Segmentos HEX0 a HEX2:

- Seleccionar: **University Program > Generic IO > Parallel Port** y pulsar en **Add**, para lanzar la ventana de configuración mostrada en la *Figura 27*.
- Establecer los parámetros tal como aparecen en la *Figura 27*.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como: *HEX2_HEX0*.
- Conectar el puerto `clk` del *HEX2_HEX0* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *HEX2_HEX0* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_parallel_port_slave` del *HEX2_HEX0* al puerto `data_master` de la *cpu*.

- Fijar la dirección de memoria base en el valor `0x1000_00b0`.
- En la fila `external_interface` del *HEX2_HEX0*, hacer click en la columna **Export** para exportar los puertos.

20 Establecer el puerto paralelo para los displays 7-Segmentos HEX3 a HEX5:

- Seleccionar: **University Program > Generic IO > Parallel Port** y pulsar en **Add**, para lanzar la ventana de configuración mostrada en la *Figura 28*.
- Establecer los parámetros tal como aparecen en la *Figura 28*.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como: *HEX5_HEX3*.
- Conectar el puerto `clk` del *HEX5_HEX3* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *HEX5_HEX3* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_parallel_port_slave` del *HEX5_HEX3* al puerto `data_master` de la *cpu*.
- Fijar la dirección de memoria base en el valor `0x1000_00c0`.
- En la fila `external_interface` del *HEX5_HEX3*, hacer click en la columna **Export** para exportar los puertos.

21 Establecer el puerto paralelo para los interruptores:

- Seleccionar: **University Program > Generic IO > Parallel Port** y pulsar en **Add**, para lanzar la ventana de configuración mostrada en la *Figura 29*.
- Establecer los parámetros tal como aparecen en la *Figura 29*.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como: *switches*.
- Conectar el puerto `clk` de los *switches* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` de los *switches* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_parallel_port_slave` de los *switches* al puerto `data_master` de la *cpu*.
- Conectar el puerto `interrupt` de los *switches* al puerto `irq` de la *cpu*.
- Fijar la dirección de memoria base en el valor `0x1000_00a0`.
- Fijar el nivel de interrupción, en la columna IRQ en el valor `2`.
- En la fila `external_interface` de los *switches*, hacer click en la columna **Export** para exportar los puertos.

22 Establecer el puerto paralelo para los pulsadores:

- Seleccionar: **University Program > Generic IO > Parallel Port** y pulsar en **Add**, para lanzar la ventana de configuración mostrada en la *Figura 30*.

- Establecer los parámetros tal como aparecen en la *Figura 30*.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como: *pushbuttons*.
- Conectar el puerto `clk` del *pushbuttons* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *pushbuttons* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_parallel_port_slave` del *pushbuttons* al puerto `data_master` de la *cpu*.
- Conectar el puerto `interrupt` del *pushbuttons* al puerto `irq` de la *cpu*.
- Fijar la dirección de memoria base en el valor `0x1000_0080`.
- Fijar el nivel de interrupción, en la columna IRQ en el valor `1`.
- En la fila `external_interface` del *pushbuttons*, hacer click en la columna **Export** para exportar los puertos.

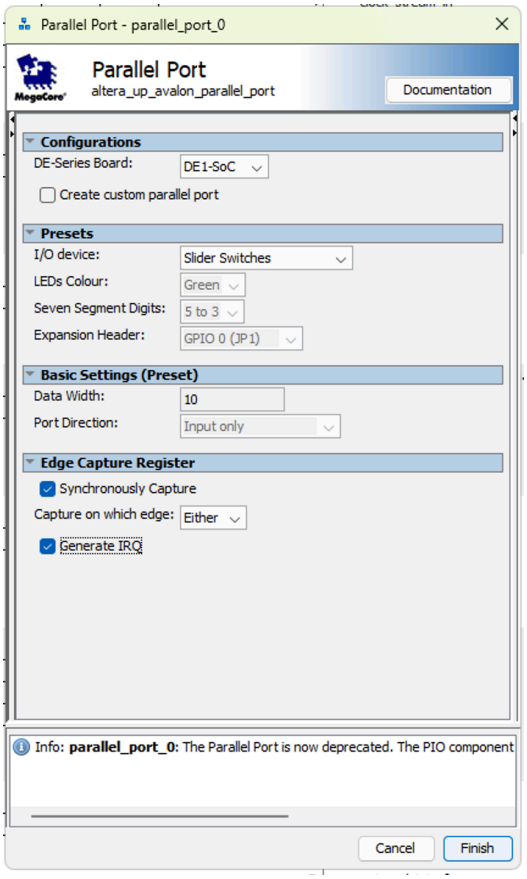

Figura 29. Definiendo las características del puerto para los interruptores.

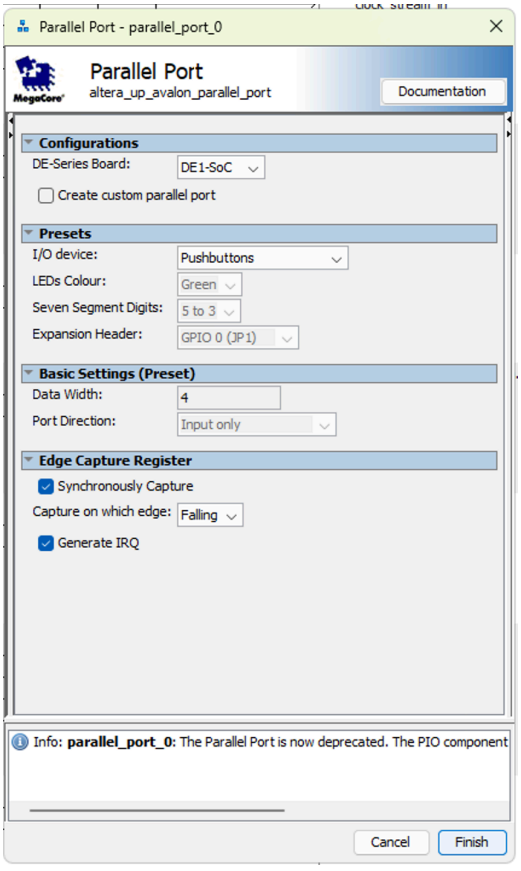

Figura 30. Definiendo las características del puerto para los pulsadores.

23 Establecer un controlador para teclado PS2:

- Seleccionar: **University Program > Generic IO > PS2 Controller** y pulsar en **Add**, para lanzar la ventana de configuración mostrada en la *Figura 31*.

- Establecer los parámetros tal como aparecen en la *Figura 31*.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como: *ps2_key*.
- Conectar el puerto `clk` del *ps2_key* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *ps2_key* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_ps2_slave` del *ps2_key* al puerto `data_master` de la *cpu*.
- Conectar el puerto `interrupt` del *ps2_key* al puerto `irq` de la *cpu*.
- Fijar la dirección de memoria base en el valor `0x1000_00d0`.
- Fijar el nivel de interrupción, en la columna IRQ en el valor `3`.
- En la fila `external_interface` del *ps2_key*, hacer click en la columna **Export** para exportar los puertos.

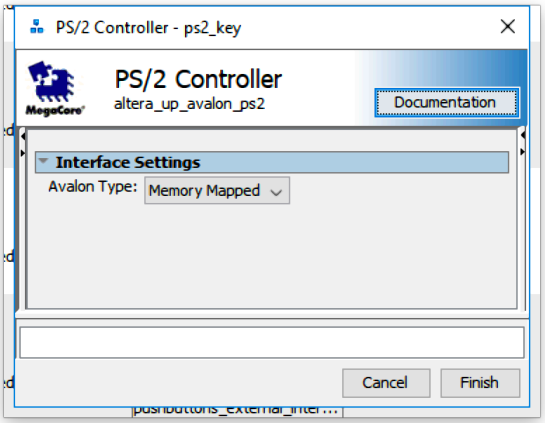

Figura 31. Definición de un puerto PS2.

24 Establecer un controlador para ratón PS2:

- Seleccionar: **University Program > Generic IO > PS2 Controller** y pulsar en **Add**, para lanzar la ventana de configuración mostrada en la *Figura 31*.
- Establecer los parámetros tal como aparecen en la *Figura 31*.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como: *ps2_mouse*.
- Conectar el puerto `clk` del *ps2_ mouse* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *ps2_ mouse* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_ps2_slave` del *ps2_ mouse* al puerto `data_master` de la *cpu*.
- Conectar el puerto `interrupt` del *ps2_ mouse* al puerto `irq` de la *cpu*.

- Fijar la dirección de memoria base en el valor `0x1000_00e0`.
- Fijar el nivel de interrupción, en la columna IRQ en el valor `4`.
- En la fila `external_interface` del *ps2_ mouse*, hacer clic en la columna **Export** para exportar los puertos.

25 Si se quiere utilizar el Codec de audio que incorpora la placa DE1-SoC, hay que añadir un módulo que permita acceder al mismo para generar y/o capturar audio externo. Pero además este códec necesita ser configurado con los parámetros adecuados para su correcto funcionamiento. Así, que para utilizar el códec necesitamos dos componentes en el sistema: el configurador de audio y el controlador de los datos. Veamos cómo se instancia el configurador:

- Seleccionar: **University Program > Audio & Video > Audio and Video Config** y pulsar en **Add**, para lanzar la ventana de configuración mostrada en la *Figura 32*.
- Establecer los parámetros tal como aparecen en la *Figura 32*.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como: *audio_config*.
- Conectar el puerto `clk` del *audio_config* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *audio_config* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_av_config_slave` del *audio_config* al puerto `data_master` de la *cpu*.
- Fijar la dirección de memoria base en el valor `0x1000_00f0`.
- En la fila `external_interface` del *audio_config*, hacer clic en la columna **Export** para exportar los puertos.

26 La instanciación del componente de controlador del audio se realizará de la siguiente manera:

- Seleccionar: **University Program > Audio & Video > Audio** y pulsar en **Add**, para lanzar la ventana de configuración mostrada en la *Figura 33*.
- Establecer los parámetros tal como aparecen en la *Figura 33*.
- Pulsando en **Finish** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como: *audio*.
- Conectar el puerto `clk` del *audio* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *audio* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_audio_slave` del *audio* al puerto `data_master` de la *cpu*.
- Conectar el puerto `interrupt` del *audio* al puerto `irq` de la *cpu*.
- Fijar la dirección de memoria base en el valor `0x1000_0100`.
- Fijar el nivel de interrupción, en la columna IRQ en el valor `5`.

- En la fila `external_interface` del *audio*, hacer clic en la columna **Export** para exportar los puertos.

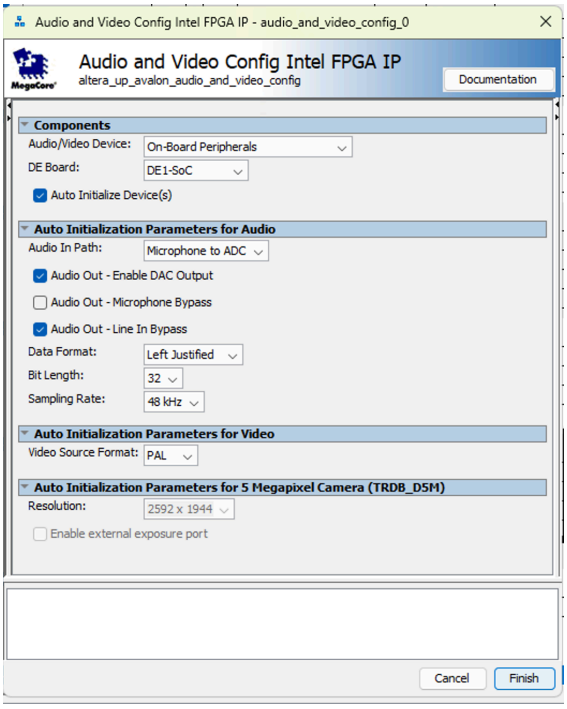

Figura 32. Definición del componente de configuración del códec de audio.

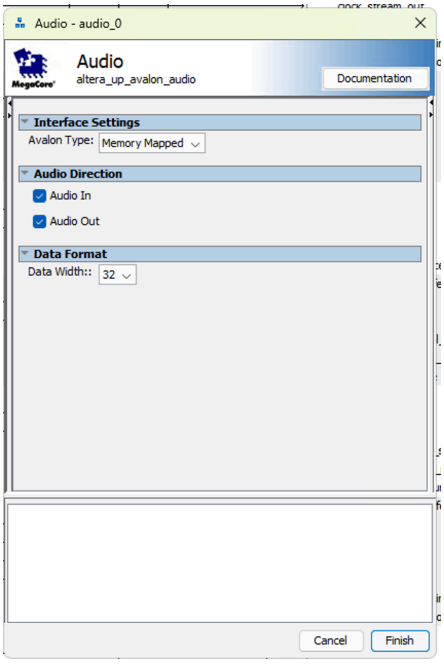

Figura 33. Definición del componente controlador del códec de audio.

27 El siguiente componente para añadir es el controlador DMA de la memoria de fondo, para el cual se procederá como sigue:

- Seleccionar: **University Program > Audio & Video > Video > DMA Controller** y pulsar en **Add**.
- En la ventana de configuración del controlador DMA, que se muestra en la *Figura 34*, se debe establecer direccionamiento X-Y, dirección del buffer en `0x03e0_0000`, una imagen de 320 × 240 pixeles en formato de 16 bits en un símbolo. El DMA debe leer la imagen de la memoria y enviarla a la pantalla, por ello se debe escoger en la dirección la opción *From Memory to Stream*.
- Pulsar el botón de **Finish** y la memoria aparecerá en ventana de contenidos del sistema.
- Renombrar el componente como *video_pixel_buffer_dma*.
- Conectar el puerto `clk` del *video_pixel_buffer_dma* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *video_pixel_buffer_dma* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_dma_master` del *video_pixel_buffer_dma* al puerto `s1` de la *sdram* y al puerto `s1` de la *onchip_memory*.
- Conectar el puerto `avalon_dma_control_slave` del *video_pixel_buffer_dma* al puerto `data_master` de la *cpu*.

- Fijar la dirección de memoria base en el valor `0x1000_0110`.

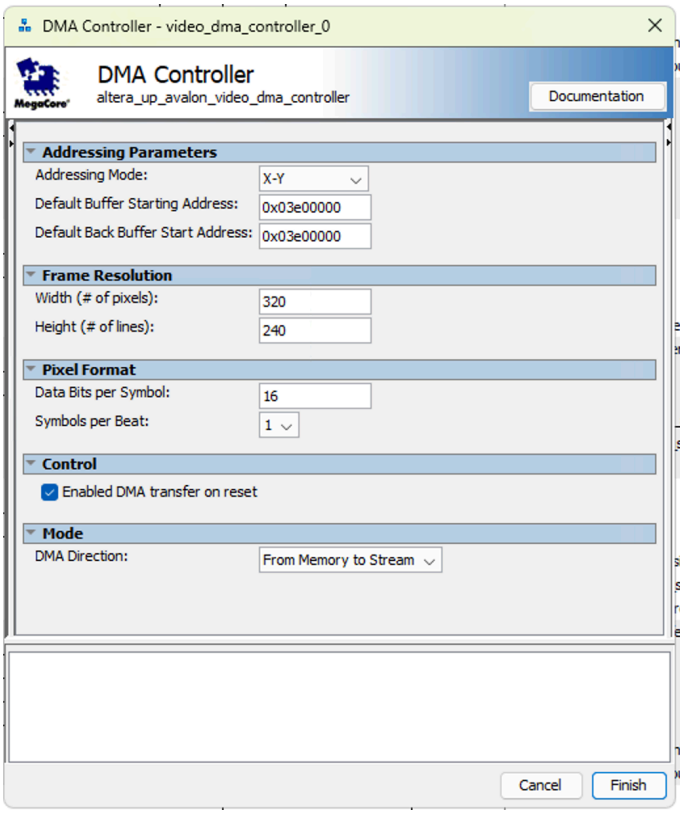

Figura 34. Definición del controlador DMA de pixeles.

28 Como se va a utilizar la misma memoria SDRAM para la ejecución del programa y como memoria de video, conviene poner una FIFO después del DMA para asegurar que se pueda mantener un flujo constante de datos con la VGA. En el diagrama de la *Figura 5*, no aparece esta FIFO, pero se encontrará ubicada entre el DMA anterior y el RGB Resampler posterior. La FIFO que se va a utilizar dispone de dos señales de reloj distintas, una para la entrada de datos y otra para la salida. En esta ocasión como su función será la regulación del tráfico de datos, ambas señales de reloj serán la misma. Para añadir la FIFO se procederá como sigue:

- Seleccionar: **University Program > Audio & Video > Video > Dual-Clock FIFO** y pulsar en **Add**.
- En la ventana de configuración del componente, que se muestra en la *Figura 35*, se debe establecer los parámetros indicados.
- Pulsar el botón de **Finish** y la memoria aparecerá en ventana de contenidos del sistema.
- Renombrar el componente como *video_buffer*.
- Conectar los puertos `clock_stream_in` y `clock_stream_out` del *video_buffer* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar los puertos `reset_stream_in` y `reset_stream_out` del *video_buffer* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.

- Conectar el puerto `avalon_dc_buffer_sink` del *video_buffer* al puerto `avalon_pixel_source` del *video_pixel_buffer_dma*.

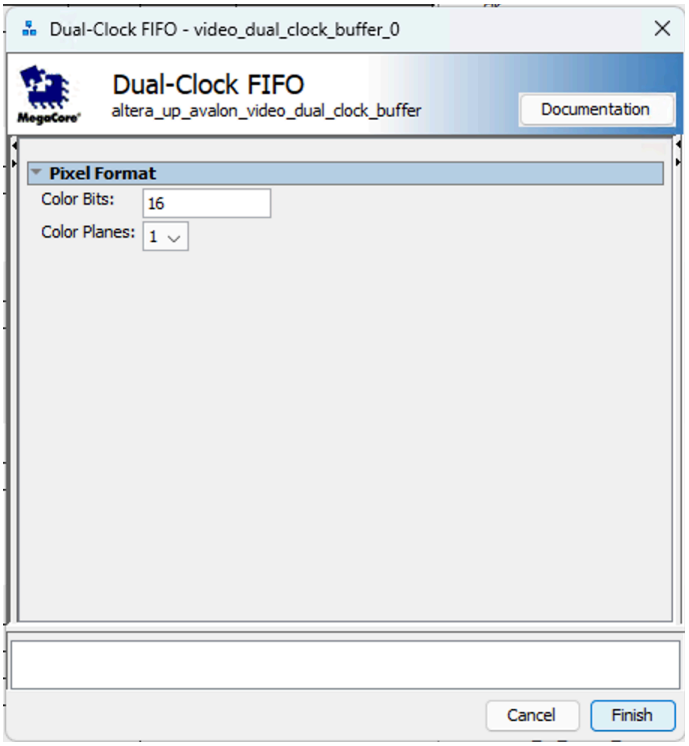

Figura 35. Definición de los parámetros de la FIFO.

29 Añadir ahora el componente RGB Resampler, para el cual se procederá como sigue:

- Seleccionar: **University Program > Audio & Video > Video > RGB Resampler** y pulsar en **Add**.
- En la ventana de configuración del RGB Resampler, que se muestra en la *Figura 36*, se debe establecer una entrada de 16 bits RGB y una salida de 30 bits RGB.
- Pulsar el botón de **Finish** y la memoria aparecerá en ventana de contenidos del sistema.
- Renombrar el componente como *video_rgb_resampler*.
- Conectar el puerto `clk` del *video_rgb_resampler* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *video_rgb_resampler* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_rgb_sink` del *video_rgb_resampler* al puerto `avalon_dc_buffer_source` del *video_buffer*.

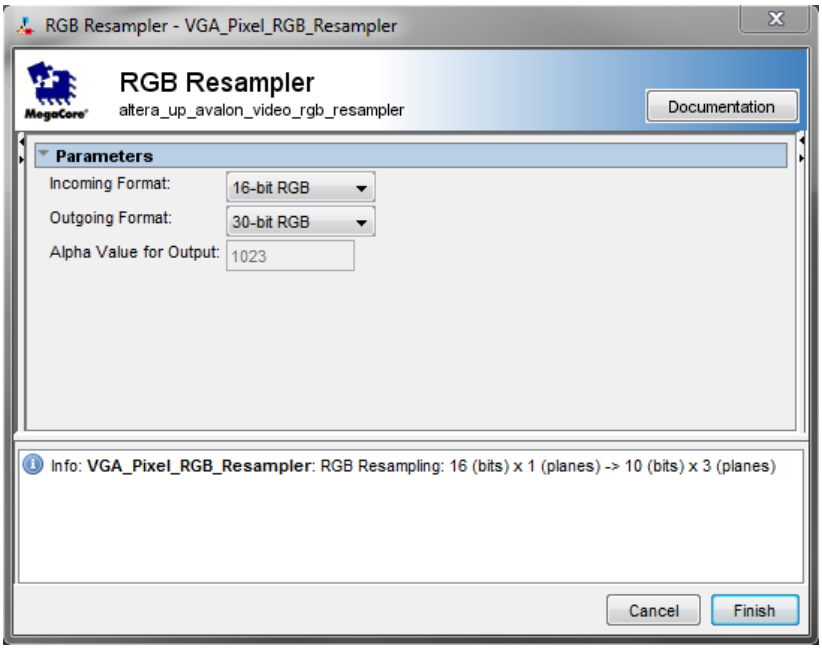

Figura 36. Definición de los parámetros del RGB Resampler.

30 Añadir ahora el componente Scaler, para el cual se procederá como sigue:

- Seleccionar: **University Program > Audio & Video > Video > Scaler** y pulsar en **Add**.
- En la ventana de configuración del componente, que se muestra en la *Figura 37*, se debe establecer los parámetros indicados.
- Pulsar el botón de **Finish** y la memoria aparecerá en ventana de contenidos del sistema.
- Renombrar el componente como *video_scaler*.
- Conectar el puerto `clk` del *video_scaler* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *video_scaler* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_scaler_sink` del *video_scaler* al puerto `avalon_rgb_source` del *video_rgb_resampler*.

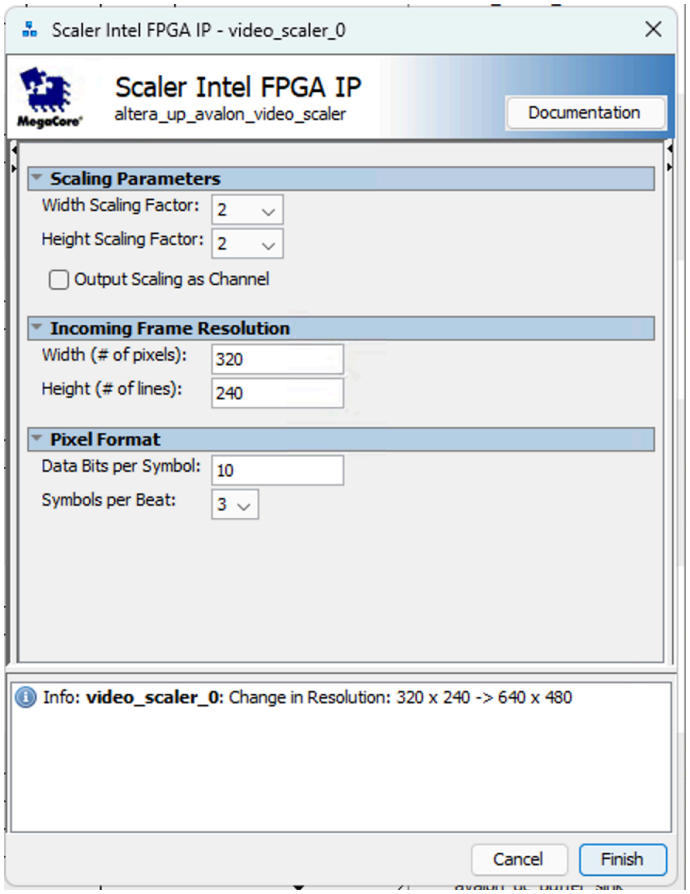

Figura 37. Definición de los parámetros del Scaler.

31 Una vez puestos todos los bloques que conforman la imagen de fondo o background, se pasará a añadir el buffer de caracteres o foreground, para lo cual se procederá como sigue:

- Seleccionar: **University Program > Audio & Video > Video > Character Buffer for VGA Display** y pulsar en **Add**.
- En la ventana de configuración del componente, que se muestra en la *Figura 38*, se debe establecer los parámetros indicados.
- Pulsar el botón de **Finish** y la memoria aparecerá en ventana de contenidos del sistema.
- Renombrar el componente como *video_char_buffer*.
- Conectar el puerto `clk` del *video_char_buffer* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *video_char_buffer* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_char_control_slave` del *video_char_buffer* al puerto `data_master` de la *cpu*.
- Conectar el puerto `avalon_char_buffer_slave` del *video_char_buffer* al puerto `data_master` de la *cpu*.
- Fijar la dirección de memoria base del puerto `avalon_char_control_slave` en el valor `0x1000_1000`.

- Fijar la dirección de memoria base del puerto `avalon_char_buffer_slave` en el valor `0x1000_2000`.

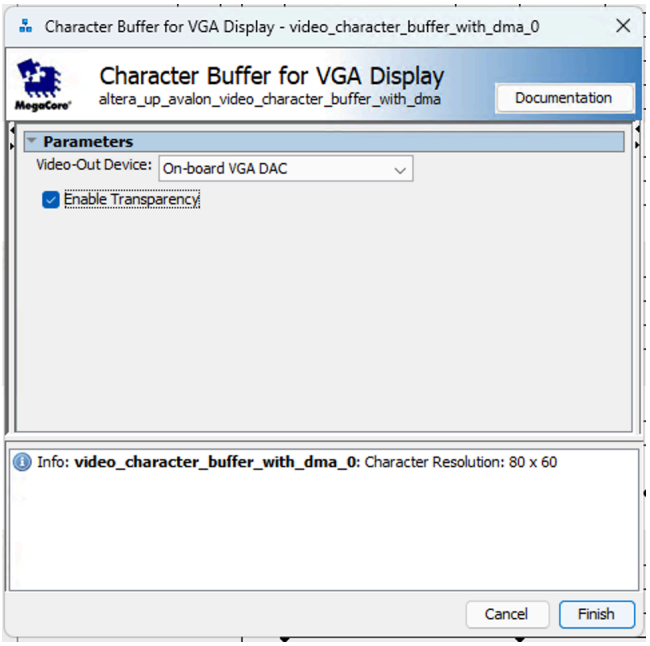

Figura 38. Definición de los parámetros del buffer de caracteres.

32 El siguiente componente será el encargado de mezclar la imagen de fondo con los caracteres, para el cual se procederá como sigue:

- Seleccionar: **University Program > Audio & Video > Video > Alpha Blender** y pulsar en **Add**.
- En la ventana de configuración del componente, que se muestra en la *Figura 39*, se debe establecer los parámetros indicados.
- Pulsar el botón de **Finish** y la memoria aparecerá en ventana de contenidos del sistema.
- Renombrar el componente como *alpha_blending*.
- Conectar el puerto `clk` del *alpha_blending* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset` del *alpha_blending* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_foreground_sink` del *alpha_blending* al puerto `avalon_char_source` de la *video_char_buffer*.
- Conectar el puerto `avalon_background_sink` del *alpha_blending* al puerto `avalon_scaler_source` del *video_scaler*.

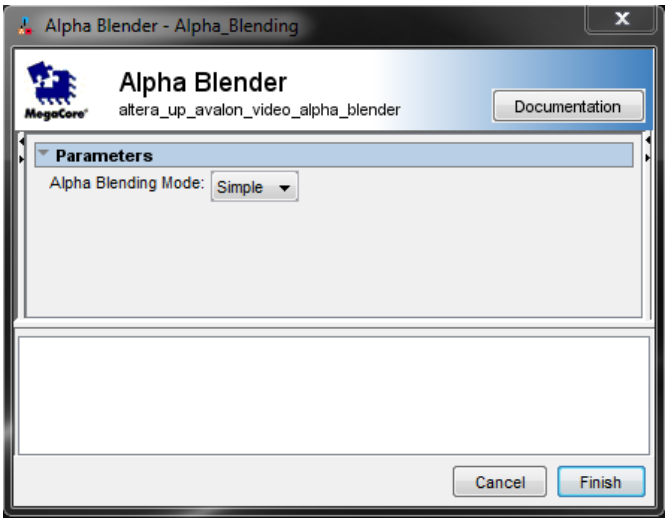

Figura 39. Definición de los parámetros del Alpha Blender.

33 Ahora se pondrá la FIFO encargada de enlazar entre el reloj del sistema de 100 MHZ y el de la pantalla VGA de 25 MHz. Conviene recordar que cuando se creó el sistema, fueron generados los relojes adecuados para esta ampliación del diseño, tal como se muestra en la *Figura 5*. Para añadir la FIFO se procederá como sigue:

- Seleccionar: **University Program > Audio & Video > Video > Dual-Clock FIFO** y pulsar en **Add**.
- En la ventana de configuración del componente, que se muestra en la *Figura 40*, se debe establecer los parámetros indicados.
- Pulsar el botón de **Finish** y la memoria aparecerá en ventana de contenidos del sistema.
- Renombrar el componente como *video_dual_clock_buffer*.
- Conectar el puerto `clock_stream_in` del *video_dual_clock_buffer* al puerto `sys_clk` del *sys_sdram_pll*.
- Conectar el puerto `reset_stream_in` del *video_dual_clock_buffer* al puerto `reset_source` del *sys_sdram_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `clock_stream_out` del *video_dual_clock_buffer* al puerto `vga_clk` del *video_pll*.
- Conectar el puerto `reset_stream_out` del *video_dual_clock_buffer* al puerto `reset_source` del *video_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_dc_buffer_sink` del *video_dual_clock_buffer* al puerto `avalon_blended_source` del *alpha_blending*.

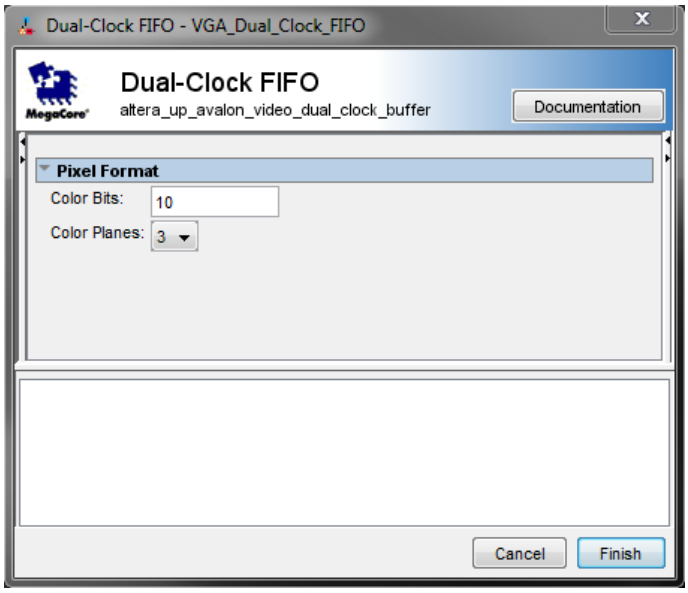

Figura 40. Definición de los parámetros de la FIFO de dos relojes.

34 Por último, solo resta añadir el controlador de la pantalla VGA, para el cual se procederá como sigue:

- Seleccionar: **University Program > Audio & Video > Video > VGA Controller** y pulsar en **Add**.
- En la ventana de configuración del componente, que se muestra en la *Figura 41*, se debe establecer los parámetros indicados.
- Pulsar el botón de **Finish** y la memoria aparecerá en ventana de contenidos del sistema.
- Renombrar el componente como *video_controller*.
- Conectar el puerto `clk` del *video_controller* al puerto `vga_clk` del *video_pll*.
- Conectar el puerto `reset` del *video_controller* al puerto `reset_source` del *video_pll* y al puerto `debug_reset_request` de la *cpu*.
- Conectar el puerto `avalon_vga_sink` del *video_controller* al puerto `avalon_dc_buffer_source` del *video_dual_clock_buffer*.
- En la fila `external_interface` del *video_controller*, hacer click en la columna **Export** para exportar los puertos del controlador de la pantalla VGA.

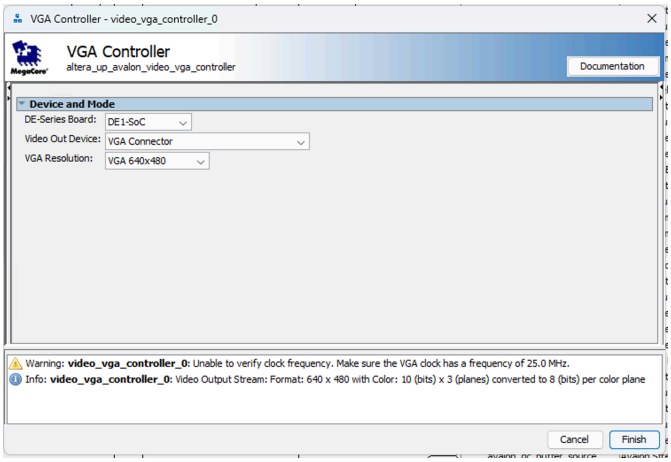

Figura 41. Definición de los parámetros del controlador de la pantalla VGA.

35 Una vez concluida la especificación de todos los componentes necesarios para implementar el sistema integrado que se desea, hay que guardar el sistema. En el menú de la *Platform Designer* **File > Save As…**, y se guarda con el nombre de *nios_system*. El guardado del sistema comenzará cuando aparezca un cuadro de dialogo donde se ve el progreso y los mensajes sobre el mismo. El proceso concluye cuando al final del cuadro de dialogo se puede leer en verde: “Save System: completed successfully”.

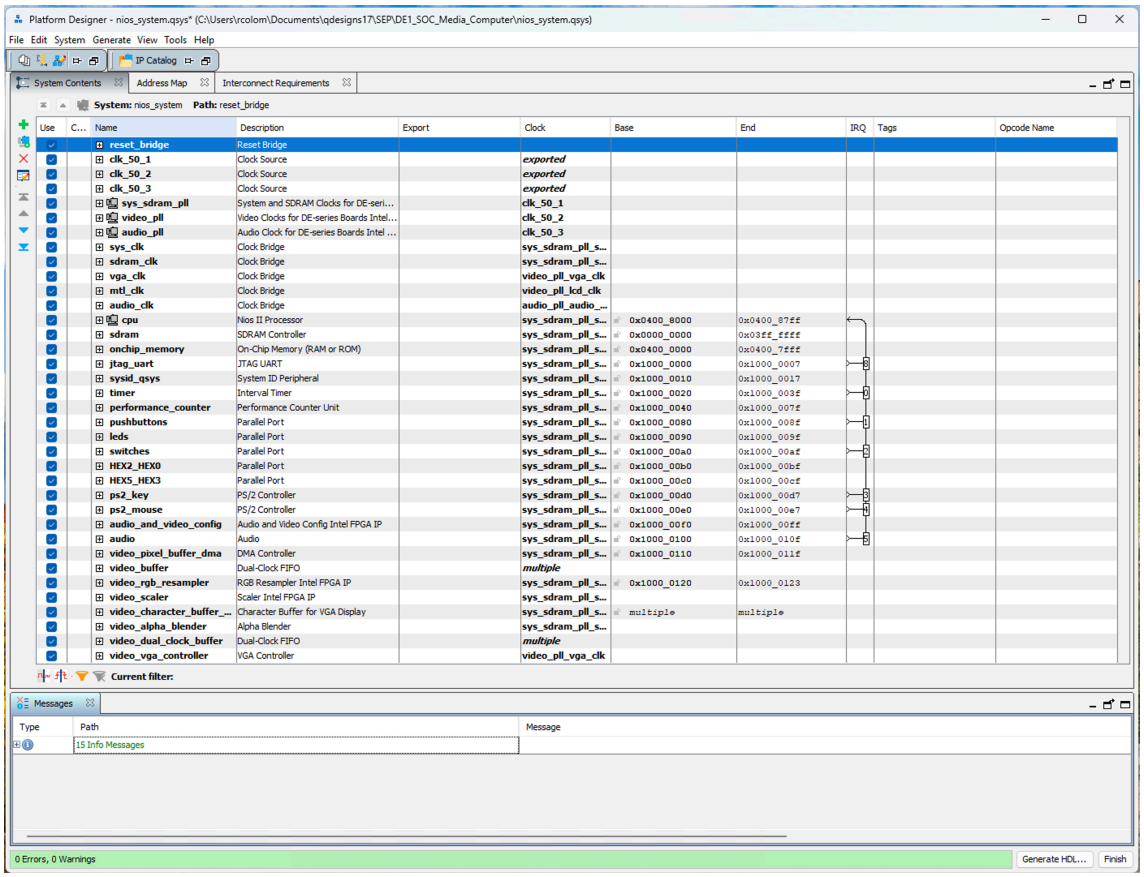

Figura 42. El sistema Nios definitivo.

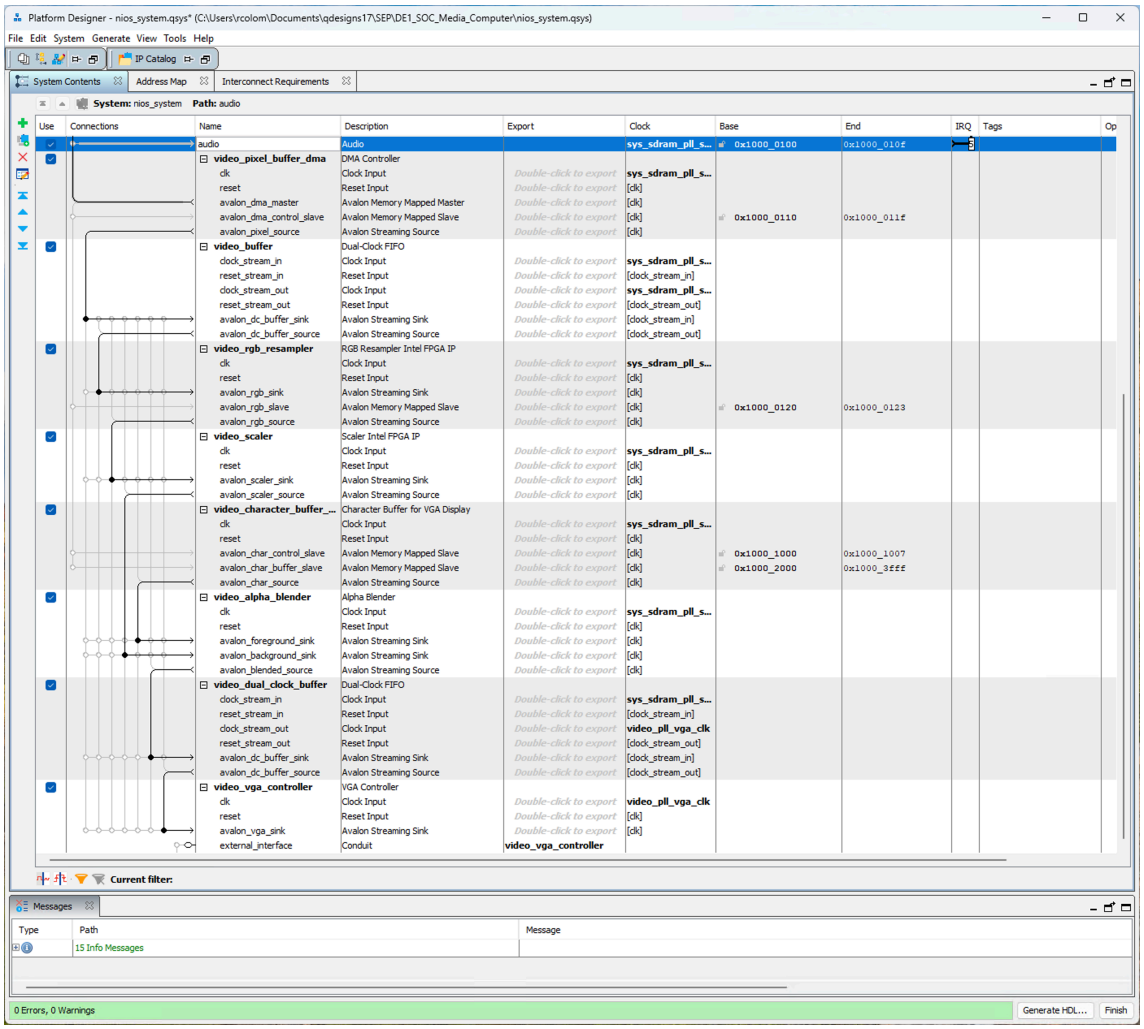

Figura 43. El sistema Nios definitivo con detalle de los bloques IP para la visualización.

36 El último paso es la generación del sistema. Para generar el sistema se debe ir al menú superior de la *Platform Designer* el comando **Generate > Generate HDL**, se abre la ventana de la *Figura 44*.

- En **Create HDL design file for synthesis**, seleccionar **Verilog**.
- Desmarcar las opciones de **Create timing and resource estimates for third-party EDA synthesis tools** y **Create block symbol file (.bsf)**.
- Seleccionar **None** para **Create simulation model**.
- Pulsar el botón **Generate**.
- La generación del sistema comenzará cuando aparezca un cuadro de dialogo donde se ve el progreso y los mensajes sobre el mismo. El proceso debe concluir sin errores, con el mensaje: *Generate: completed successfully*.
- Pulsar en el **Close** de la ventana.
- No cerrar, de momento, la *Platform Designer* y volver a Quartus Prime.

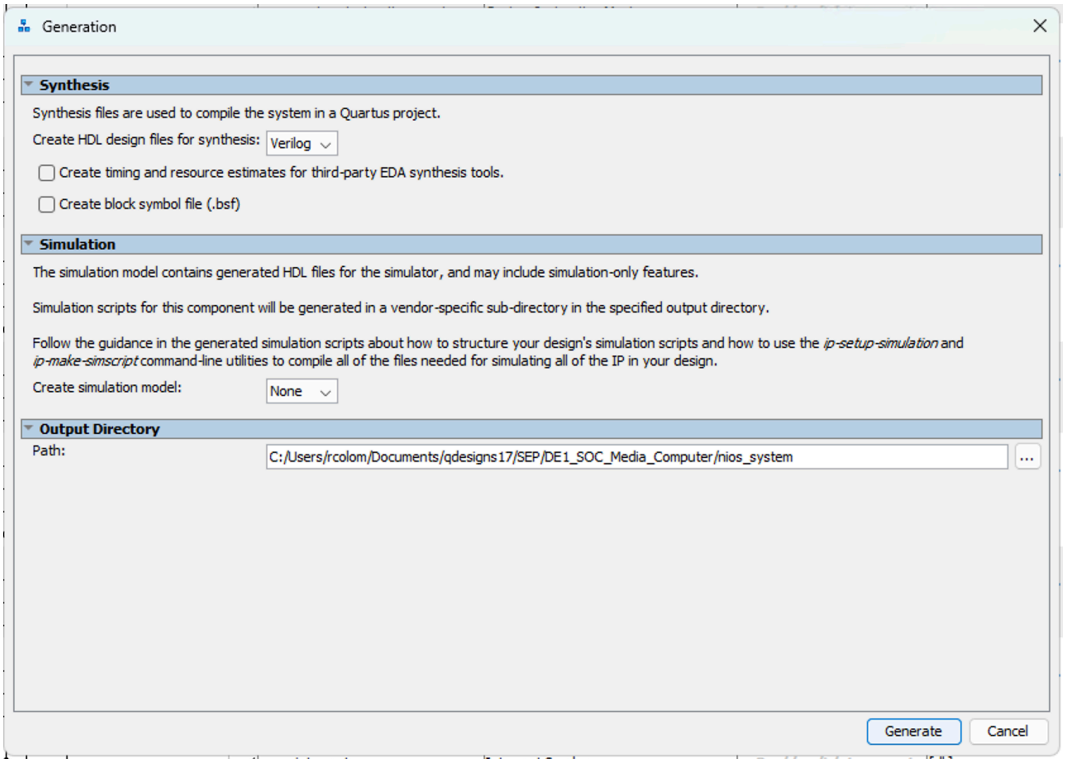

Figura 44. Generación del sistema Nios.

8. Integración del Sistema Nios II en el proyecto de Quartus Prime

Para completar el diseño del hardware, se deben realizar las tareas que a continuación se relatan:

- Instanciación del sistema Nios II creado mediante el *Platform Designer* en el proyecto de Quartus Prime.
- Asignaciones de los pines de la FPGA.
- Compilación del proyecto de Quartus Prime.
- Configuración de la FPGA.

8.1. Instanciación del Módulo Generado por el *Platform Designer*

La instanciación del módulo generado depende del método escogido como entrada de diseño para Quartus Prime. Se puede escoger entre esquemático, Verilog y VHDL. En este apartado se verá cómo realizar la instanciación utilizando el lenguaje HDL Verilog.

Cuando la *Platform Designer* genera el sistema, crea una serie de ficheros de descripción de ese sistema. Estos ficheros se realizan en lenguaje Verilog o en VHDL dependiendo de la selección que se haya escogido al crear el nuevo sistema en la *Platform Designer*. En cualquiera de los casos, en el menú de la *Platform Designer* hay una opción denominada **Generate > Show Instantiation Template…**, donde se puede obtener la descripción del sistema en el lenguaje que se desee. Esta descripción es el que se debe tomar como referencia para realizar la instanciación del componente mediante un lenguaje HDL.

```
//=======================================================
// Ricardo J. Colom
// Diciembre 2024
//=======================================================

module DE1_SoC_Media_Computer(

	//////////// ADC //////////
	output		          		ADC_CONVST,
	output		          		ADC_DIN,
	input 		          		ADC_DOUT,
	output		          		ADC_SCLK,

	//////////// Audio //////////
	input 		          		AUD_ADCDAT,
	inout 		          		AUD_ADCLRCK,
	inout 		          		AUD_BCLK,
	output		          		AUD_DACDAT,
	inout 		          		AUD_DACLRCK,
	output		          		AUD_XCK,

	//////////// CLOCK //////////
	input 		          		CLOCK2_50,
	input 		          		CLOCK3_50,
	input 		          		CLOCK4_50,
	input 		          		CLOCK_50,

	//////////// SDRAM //////////
	output		    [12:0]		DRAM_ADDR,
	output		     [1:0]		DRAM_BA,
	output		          		DRAM_CAS_N,
	output		          		DRAM_CKE,
	output		          		DRAM_CLK,
	output		          		DRAM_CS_N,
	inout 		    [15:0]		DRAM_DQ,
	output		          		DRAM_LDQM,
	output		          		DRAM_RAS_N,
	output		          		DRAM_UDQM,
	output		          		DRAM_WE_N,

	//////////// I2C for Audio and Video-In //////////
	output		          		FPGA_I2C_SCLK,
	inout 		          		FPGA_I2C_SDAT,

	//////////// SEG7 //////////
	output		     [6:0]		HEX0,
	output		     [6:0]		HEX1,
	output		     [6:0]		HEX2,
	output		     [6:0]		HEX3,
	output		     [6:0]		HEX4,
	output		     [6:0]		HEX5,

	//////////// IR //////////
	input 		          		IRDA_RXD,
	output		          		IRDA_TXD,

	//////////// KEY //////////
	input 		     [3:0]		KEY,

	//////////// LED //////////
	output		     [9:0]		LEDR,

	//////////// PS2 //////////
	inout 		          		PS2_CLK,
	inout 		          		PS2_CLK2,
	inout 		          		PS2_DAT,
	inout 		          		PS2_DAT2,

	//////////// SW //////////
	input 		     [9:0]		SW,

	//////////// Video-In //////////
	input 		          		TD_CLK27,
	input 		     [7:0]		TD_DATA,
	input 		          		TD_HS,
	output		          		TD_RESET_N,
	input 		          		TD_VS,

	//////////// VGA //////////
	output		          		VGA_BLANK_N,
	output		     [7:0]		VGA_B,
	output		          		VGA_CLK,
	output		     [7:0]		VGA_G,
	output		          		VGA_HS,
	output		     [7:0]		VGA_R,
	output		          		VGA_SYNC_N,
	output		          		VGA_VS
);

//=======================================================
//  REG/WIRE declarations
//=======================================================

assign TD_RESET_N  	= 1'b1;

//=======================================================
//  Structural coding
//=======================================================

// Aquí es donde va vuestro Sistema generado en la Platform Designer

endmodule
```

Figura 45. Ejemplo de instanciación del sistema Nios II con Verilog.

En primer lugar, hay que instanciar el módulo del sistema Nios. El método de entrada de diseño escogido en este caso será mediante el HDL Verilog. Para lo cual hay que seguir los siguientes pasos:

43 En proyecto de Quartus Prime `DE1_SoC_Media_Computer`, creado anteriormente, abrir un nuevo editor de textos. En el menú **File > New** se abre una ventana donde se debe seleccionar **Verilog HDL File** y después **OK**.

44 Crear un "**module**" de Verilog nombrado como `DE1_SoC_Media_Computer`, que tenga las entradas y salidas de la placa DE1-SoC, tal como se muestra en la *Figura 45*; o bien descargar de **PoliformaT** el archivo `DE1_SoC_Media_Computer.v`, que contiene ya la descripción.

45 Para instanciar el módulo de Nios II mediante Verilog, hay que conocer como ha sido creado dicho módulo. Para ello en el menú de la *Platform Designer* hay una opción denominada **Generate > Show Instantiation Template…**, donde se puede obtener la información sobre la instanciación del módulo mediante HDLs (Verilog o VHDL), tal como se muestra en la *Figura 46*. El código HDL se puede copiar fácilmente en el editor de Quartus Prime sin más que pulsar el botón de **Copy** que hay en la parte inferior de la ventana.

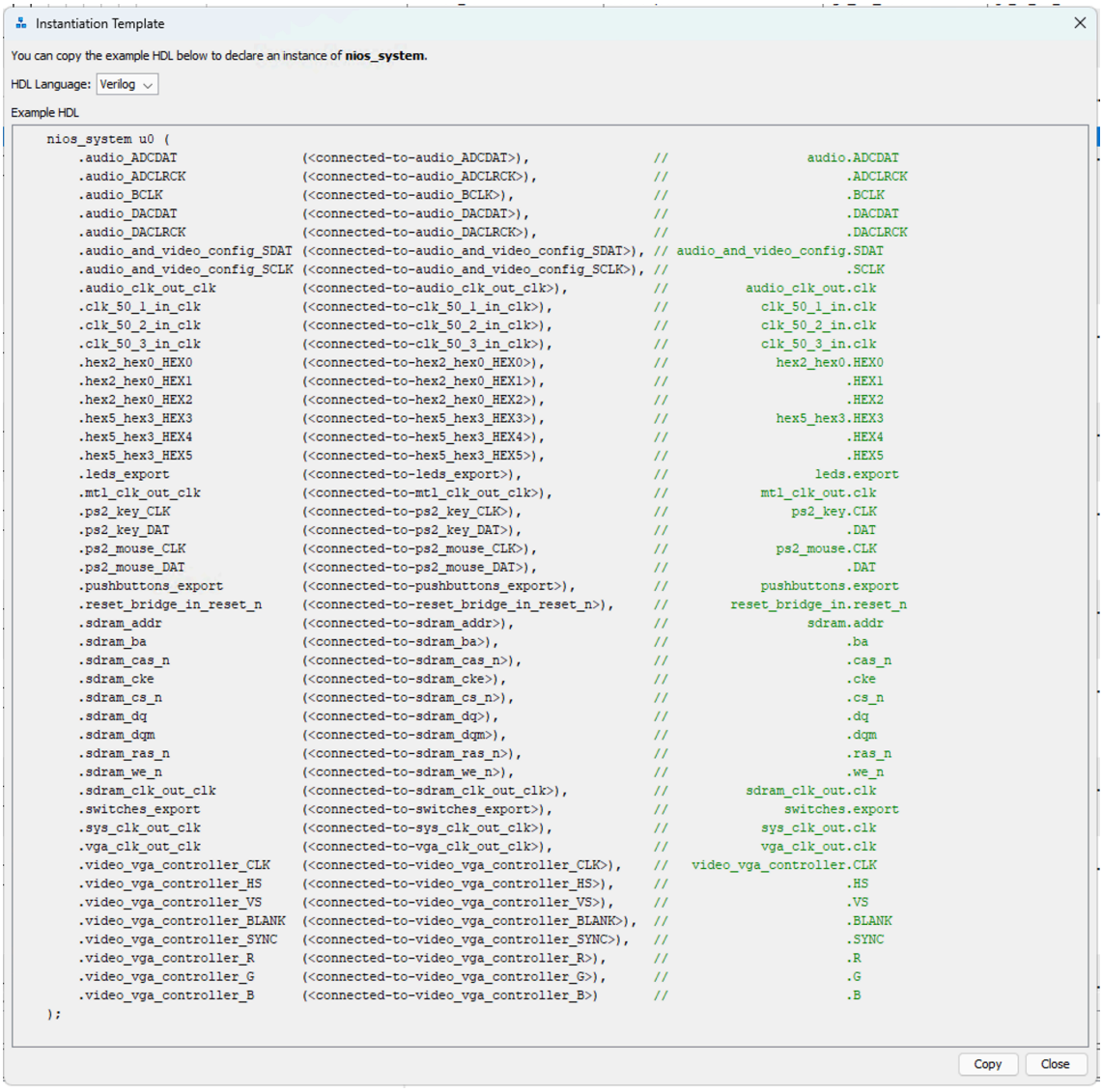

Figura 46. Código HDL del módulo Nios creado por La Platform Designer.

46 Pegar el código HDL de la instancia copiada de la *Platform Designer* en la parte correspondiente del archivo `DE1_SoC_Media_Computer.v`, donde pone "**`//Aquí es donde va vuestro Sistema generado en la Platform Designer`**", en la *Figura 45*.

47 Conectar adecuadamente los puertos de entrada salida con las señales de la instancia `nios_system`. El resultado final se muestra en la *Figura 47*.

```
nios_system u0 (
        .clk_50_1_in_clk              (CLOCK_50),                  //           clk_50_1_in.clk
        .reset_bridge_in_reset_n      (1'b1),                      //       reset_bridge_in.reset_n
        .clk_50_2_in_clk              (CLOCK2_50),                 //           clk_50_2_in.clk
        .clk_50_3_in_clk              (CLOCK3_50),                 //           clk_50_3_in.clk
        .sys_clk_out_clk              (),                          //           sys_clk_out.clk
        .sdram_clk_out_clk            (DRAM_CLK),                  //         sdram_clk_out.clk
        .vga_clk_out_clk              (),                          //           vga_clk_out.clk
        .mtl_clk_out_clk              (),                          //           mtl_clk_out.clk
        .audio_clk_out_clk            (AUD_XCK),                   //         audio_clk_out.clk
        .sdram_addr                   (DRAM_ADDR),                 //                 sdram.addr
        .sdram_ba                     (DRAM_BA),                   //                       .ba
        .sdram_cas_n                  (DRAM_CAS_N),                //                       .cas_n
        .sdram_cke                    (DRAM_CKE),                  //                       .cke
        .sdram_cs_n                   (DRAM_CS_N),                 //                       .cs_n
        .sdram_dq                     (DRAM_DQ),                   //                       .dq
        .sdram_dqm                    ({DRAM_UDQM, DRAM_LDQM}),    //                       .dqm
        .sdram_ras_n                  (DRAM_RAS_N),                //                       .ras_n
        .sdram_we_n                   (DRAM_WE_N),                 //                       .we_n
        .pushbuttons_export           (KEY),                       //           pushbuttons.export
        .leds_export                  (LEDR),                      //                  leds.export
        .switches_export              (SW),                        //              switches.export
        .hex2_hex0_HEX0               (HEX0),                      //             hex2_hex0.HEX0
        .hex2_hex0_HEX1               (HEX1),                      //                       .HEX1
        .hex2_hex0_HEX2               (HEX2),                      //                       .HEX2
        .hex5_hex3_HEX3               (HEX3),                      //             hex5_hex3.HEX3
        .hex5_hex3_HEX4               (HEX4),                      //                       .HEX4
        .hex5_hex3_HEX5               (HEX5),                      //                       .HEX5
        .ps2_key_CLK                  (PS2_CLK),                   //               ps2_key.CLK
        .ps2_key_DAT                  (PS2_DAT),                   //                       .DAT
        .ps2_mouse_CLK                (PS2_CLK2),                  //             ps2_mouse.CLK
        .ps2_mouse_DAT                (PS2_DAT2),                  //                       .DAT
        .audio_and_video_config_SDAT  (FPGA_I2C_SDAT),             // audio_and_video_config.SDAT
        .audio_and_video_config_SCLK  (FPGA_I2C_SCLK),             //                       .SCLK
        .audio_ADCDAT                 (AUD_ADCDAT),                //                audio.ADCDAT
        .audio_ADCLRCK                (AUD_ADCLRCK),               //                       .ADCLRCK
        .audio_BCLK                   (AUD_BCLK),                  //                       .BCLK
        .audio_DACDAT                 (AUD_DACDAT),                //                       .DACDAT
        .audio_DACLRCK                (AUD_DACLRCK),               //                       .DACLRCK
        .video_vga_controller_CLK     (VGA_CLK),                   //   video_vga_controller.CLK
        .video_vga_controller_HS      (VGA_HS),                    //                       .HS
        .video_vga_controller_VS      (VGA_VS),                    //                       .VS
        .video_vga_controller_BLANK   (VGA_BLANK_N),               //                       .BLANK
        .video_vga_controller_SYNC    (VGA_SYNC_N),                //                       .SYNC
        .video_vga_controller_R       (VGA_R),                     //                       .R
        .video_vga_controller_G       (VGA_G),                     //                       .G
        .video_vga_controller_B       (VGA_B)                      //                       .B
);
```

Figura 47. Ejemplo de instanciación del sistema Nios II con Verilog. Conexión Puertos.

48 Guardar el archivo Verilog como `DE1_SoC_Media_Computer.v`.

49 Si en estas condiciones se intenta realizar una compilación del diseño, Quartus Prime dará el error que no encuentra el `nios_system`. Por tanto, hay que añadírselo a los archivos del proyecto.

50 En el menú **Assignments > Settings** se abre una nueva ventana. En **Category**, pulsar en **Files**. Ahí debe aparecer el archivo de Verilog `DE1_SoC_Media_Computer.v`. Ahora en **File**

name pulsar sobre el botón **(…)**, en la nueva ventana seleccionar como tipo de fichero **Script Files (*.tcl, *.sdc, *.qip)**. Localizar el archivo: <*directorio de trabajo*>**/nios_system/synthesis/nios_system.qip** y pulsar en **Open** para seleccionar el archivo. Luego pulsar **Add** para añadir el fichero al proyecto y terminar pulsando el **OK**.

51 Ahora ya se puede compilar el diseño sin errores.

8.2. Asignación de Pines

A la hora de realizar la asignación de los pines para la placa DE1-SoC existen dos posibilidades. Realizar manualmente las asignaciones, lo cual bastante entretenido, o la opción más comoda y sencilla, consistente en importar el fichero de asignaciones que el fabricante de la placa DE1-SoC suministra, `DE1_SoC_pin_assignments.csv`. La importación del fichero de asignaciones se realiza mediante: **Assignments > Import Assignments**.

8.3. Compilación del Proyecto de Quartus Prime

En este punto se está preparado para compilar el proyecto de Quartus Prime y verificar si el diseño final cumple los requerimientos temporales. Se debe compilar el diseño hardware para crear el fichero de configuración de la FPGA, que será descargado en la placa. Una vez haya concluido la compilación se deben analizar las características temporales del diseño para verificar si este puede funcionar adecuadamente en el hardware. El procedimiento será el siguiente:

51 En el menú **Processing** hacer clic en **Start Compilation**.

52 En la ventana de Quartus Prime de Status se visualiza el progreso. La compilación puede tardar algunos minutos. Cuando la compilación termine continuar con el siguiente punto.

53 En Quartus Prime se mostrará la ventana del informe de compilación.

En este momento concluye la integración del sistema Nios II en el proyecto de Quartus Prime. Ahora se está preparado para descargar el fichero de configuración en la placa DE1-SoC.

8.4. Descargar el Diseño Hardware en la FPGA

La programación y configuración de la Cyclone V en modo JTAG se realiza de la siguiente manera:

54 Asegurarse que la placa DE1-SoC está conectada al PC por medio de un cable USB, desde el puerto USB-Blaster de la placa. Conectar la alimentación de la placa DE1-SoC.

55 En el software de Quartus Prime seleccionar: **Tools > Programmer**, para que aparezca la ventana de la *Figura 48*.

56 Si junto al botón de **Hardware Setup** aparece la indicación de **No Hardware**, hay que pulsar en dicho botón, para indicar la conexión con el hardware, tras lo cual aparecerá el termino **DE-SoC**, tal como se aprecia en la *Figura 48*.

57 Si el modo **JTAG** no aparece seleccionado en el recuadro **Mode**, habrá que seleccionarlo. Si por defecto no aparece seleccionado el DE-SoC, hay que pulsar en **Hardware Setup** y seleccionar DE-SoC.

58 Pulsar ahora el botón **Auto Detect** de la *Figura 48*. Aparecerá una nueva ventana en la que se debe seleccionar el dispositivo **5CSEMA5**. Tras lo cual se visualizarán en la ventana de la *Figura 48* dos dispositivos: el SOCVHPS y el 5CSEMA5.

59 El fichero de configuración `DE1_SoC_Media_Computer.sof` debe asignarse al dispositivo 5CSEMA5. Para ello se selecciona con el ratón la fila correspondiente a dicho dispositivo y se pulsa el botón **Change File**. Esto permite buscar el archivo

`DE1_SoC_Media_Computer.sof` entre las carpetas del proyecto para vincularlo al dispositivo tal como se muestra en la *Figura 48*.

60 La casilla **Program/Configure** debe ser marcada.

61 En este punto la ventana del programador debe aparecer tal como se muestra en la *Figura 48*. Solo hay que pulsar el botón **Start** para realizar la configuración de la FPGA.

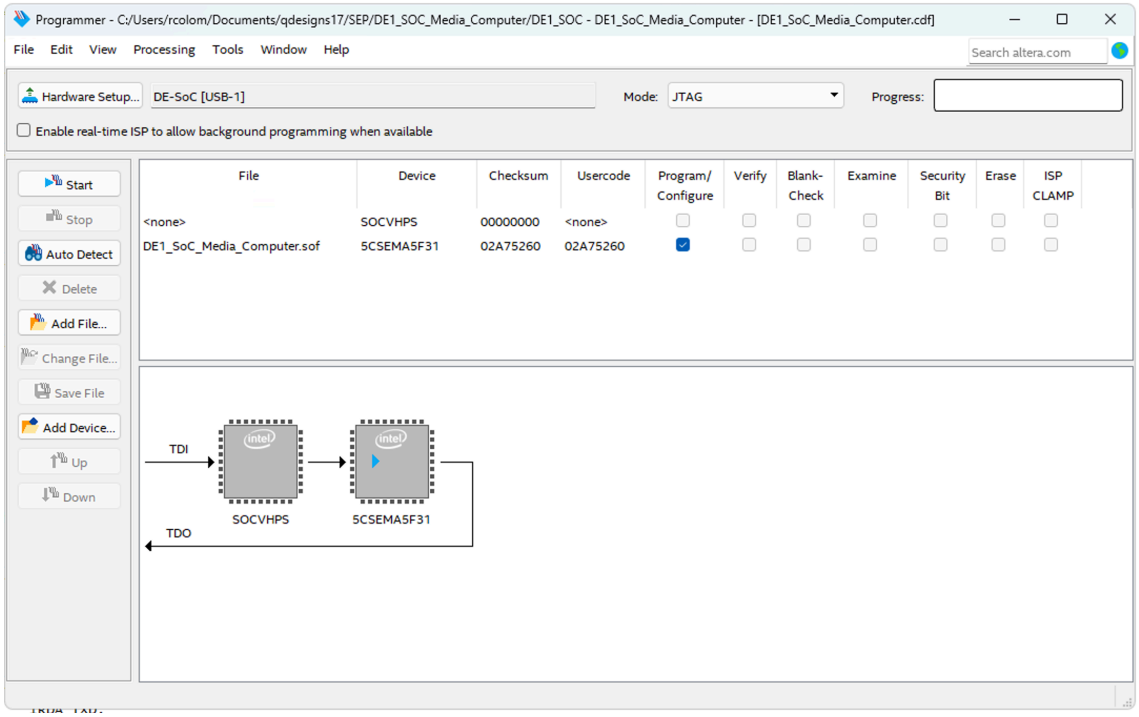

Figura 48. La ventana del programador.

9. Desarrollo y Ejecución del Programa de Aplicación

Una vez ha sido configurada la FPGA con el hardware diseñado, es necesario crear y ejecutar un programa de aplicación que permita testear los componentes de la placa DE1-SoC, para verificar que funcionan correctamente. El programa que se pide se puede realizar mediante lenguaje ensamblador para Nios II o mediante un lenguaje de alto nivel como C. A continuación, se mostrará cómo realizar la programación utilizando C/C++.

9.1. Crear un Proyecto para una Nueva Aplicación C/C++

En este apartado se utiliza el Nios II SBT (Software Build Tools) para compilar un sencillo programa en C/C++. Se mostrarán los pasos básicos del software para demostrar como ejecutar un programa en el sistema hardware desarrollado previamente. En primer lugar, para lanzar el entorno de trabajo Nios II SBT, se puede hacer de varios modos: desde Quartus Prime en **Tools > Nios II Software Build Tools for Eclipse**, desde la *Platform Designer* en **Tools > Nios II Software Build Tools for Eclipse**, desde el Menú de Inicio de Windows o desde el icono del escritorio **Nios II Software Build Tools for Eclipse** (si lo hay). En cualquier caso, debe aparecer la caja de dialogo del **Workspace Launcher**, pulsando el botón de **Browse…** se buscará el subdirectorio de trabajo de Quartus Prime, donde está el proyecto del sistema Nios y se creará una nueva carpeta denominada software. Una vez especificado como **Workspace:** *<directorio de trabajo>***/software**, hacer clic en **OK** para aceptar la

ubicación del espacio de trabajo. Una vez abierto el entorno Nios II SBT, hay que crear un proyecto de una nueva aplicación para los cual se seguirán los siguientes pasos:

62 En el menú **File > New**, clic en **Nios II Application and BSP from Template**, para crear un nuevo proyecto.

63 En el **Targer hardware information**, junto al **SOPC Information File Name** pulsar el botón **(…)**, para establecer el fichero que contiene las características del sistema. Se abrirá una caja de dialogo.

64 Buscar el directorio donde están los ficheros del diseño hardware.

65 Seleccionar el fichero `nios_system.sopcinfo`.

66 Clic en **Open**. Así se retorna a la ventana de **Nios II Application and BSP from Template**, y ahora se puede ver que los campos del **SOPC Information File Name** y de **CPU** se han rellenado.

67 En el apartado **Application project**, escribir en el **Project name** un nombre para el proyecto software, por ejemplo: `led_intermitente`.

68 Seleccionar **Blank Project** en la lista de **Project Template**. En la *Figura 49* se muestra la ventana con los datos rellenados.

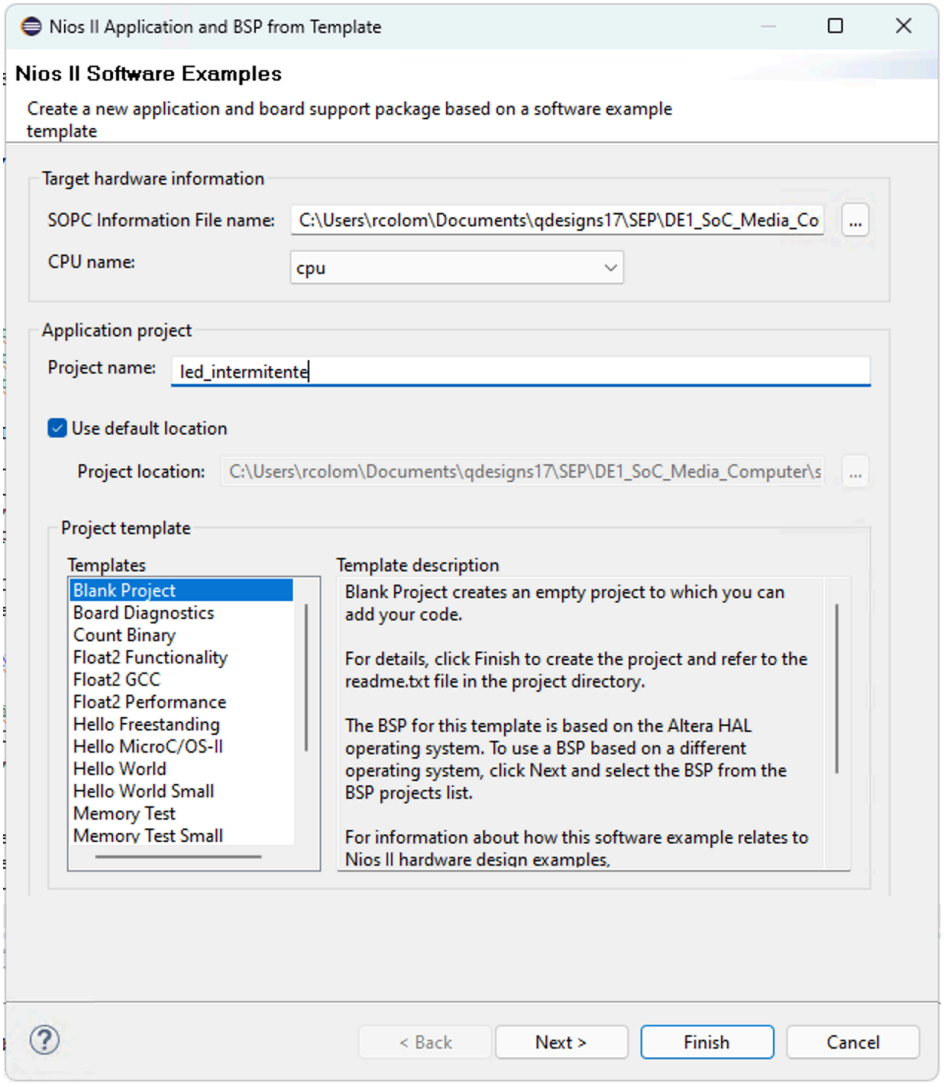

Figura 49. La creación de un nuevo proyecto en el IDE de Nios II.

69 Hacer clic en **Finish**.

El Nios II SBT ahora muestra dos nuevos proyectos en la pestaña **Project Explorer** situada en el lado izquierdo del área de trabajo: `led_intermitente` y `led_intermitente_bsp`. El proyecto `led_intermitente` es el proyecto de la aplicación C/C++ que se va a realizar, mientras que `led_intermitente_bsp` es una librería del sistema que contiene detalles del hardware del sistema Nios II.

Una vez creado el proyecto hace falta disponer de una ventana en blanco donde escribir el código C de la aplicación, para ello sobre el proyecto `led_intermitente`, pulsamos el botón derecho del ratón y en **New** se hace clic en **Source File**, esto abre una ventana en la que se indicará el nombre del fichero que se quiere crear, `led_intermitente.c`. En la ventana central del área de trabajo aparece una pestaña con el título `led_intermitente.c`, donde se debe escribir el código C mostrado en la *Figura 50*.

En el Nios II SBT se pueden editar o escribir los ficheros de programa y posteriormente realizar la compilación y la ejecución que a continuación se describen.

```
/*
 * led_intermitente.c
 *
 *  Created on: 18 de dic. de 2024
 *      Author: rcolom
 */

#include <stdio.h>
#include "system.h"
#include <io.h>

int main()
{
      printf("Hola desde Nios II!\n");
      int count = 0;
      int delay;
      while(1)
      {
            IOWR(LED_PIO_BASE, 0x0, count & 0x01);
            delay = 0;
            while(delay<2000000)
            {
                  delay++;
            }
            count++;
      }
      return 0;
}
```

Figura 50. Código C para el led intermitente.

9.2. Compilación del Proyecto

La compilación del proyecto tiene como finalidad obtener una imagen del software ejecutable, el conocido como archivo `.elf`. Se procederá de la siguiente manera:

70 Construir el proyecto: **Project > Build Project**.

71 La compilación dará el error: `'LED_PIO_BASE' undeclared`. Esto se debe a que se está intentando escribir en el puerto de los LEDs, pero el nombre no es correcto.

72 En el proyecto `led_intermitente_bsp` se encuentra el fichero `system.h`, que contiene las definiciones para el uso del sistema Nios creado por la Platform Designer. En él se puede descubrir que el nombre correcto del puerto de los LED.

73 En el código C cambiar `LED_PIO_BASE` por el correspondiente nombre de los LED indicado en el fichero `system.h`

74 Guardar el archivo.

75 Volver a compilar el proyecto: **Project > Build Project**.

9.3. Ejecución del Programa

Para descargar el software ejecutable en la placa DE1-SoC, se seguirán los siguientes pasos:

76 Haciendo clic en el botón derecho del ratón sobre el proyecto `led_intermitente`, ponerse encima de **Run As** y seleccionar **Nios II Hardware**. El Nios II SBT descargará el programa en la FPGA y comenzará la ejecución. Si aparece la ventana de **Run Configurations**, hay que verificar que **Project Name** y el **ELF file name** tiene la información necesaria (ver *Figura 51*), además puede ocurrir que no encuentre la placa, en cuyo caso hay que ir a la pestaña **Target Connection** y pulsar el botón **Refresh Connections** (ver *Figura 52*). Después se pulsa **Run**, para que descargue el software sobre la placa.

77 Cuando el hardware comience la ejecución del programa, en la pestaña **Nios II Console** del Nios II SBT se mostrará el mensaje de salida `Hola desde Nios II!`. Mientras que en la placa DE1-SoC, habrá un diodo led rojo intermitente.

78 Hacer clic en el botón de **Terminate** (cuadrado rojo) ubicado en el lado derecho de la barra de herramientas superior de la pestaña **Nios II Console**, para terminar la sesión de ejecución. Cuando se hace clic en **Terminate**, el Nios II SBT desconecta la comunicación con la placa y deja el procesador Nios II ejecutando libremente.

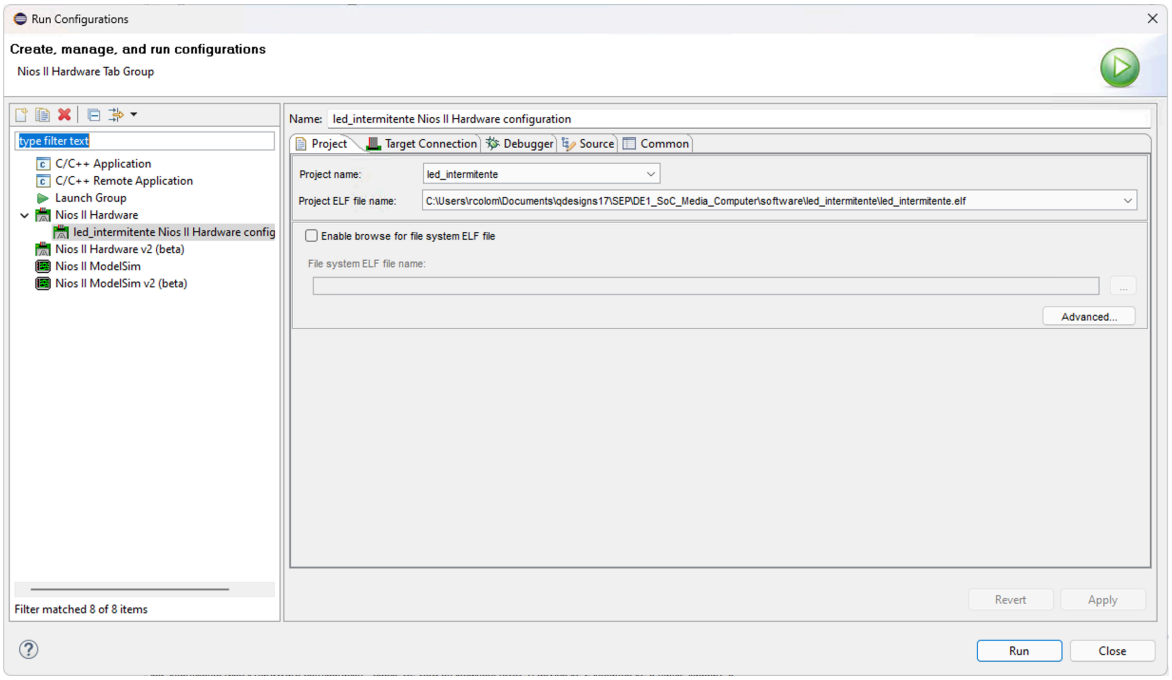

Figura 51. Ventana de configuración de la ejecución en placa.

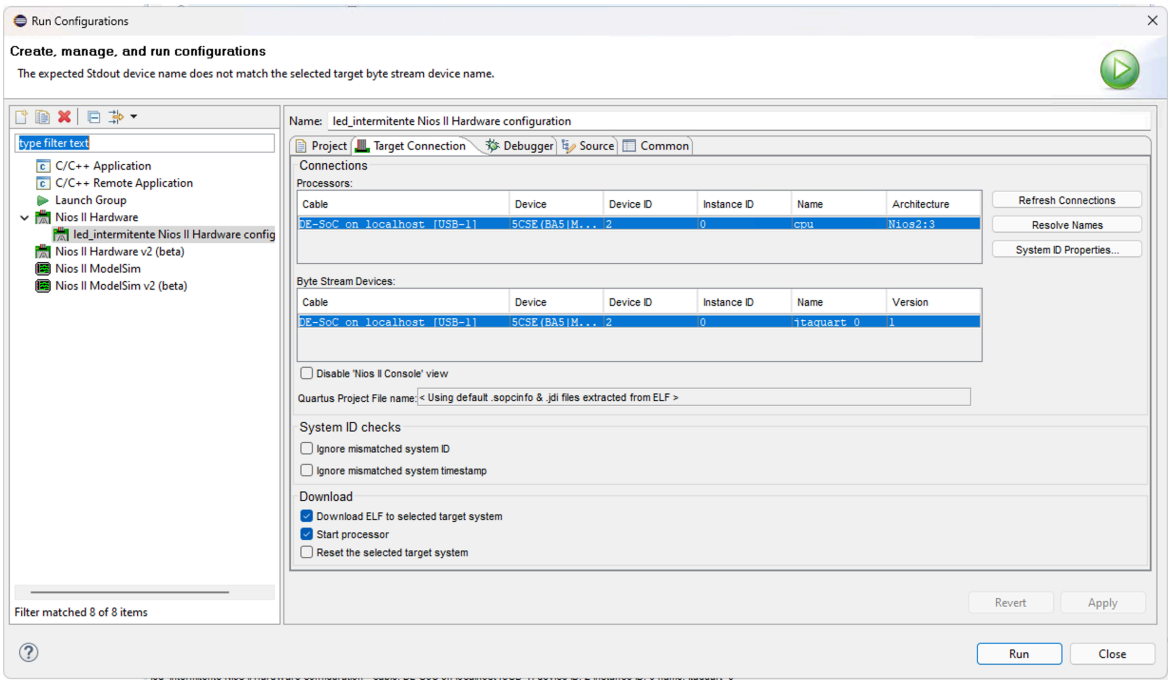

Figura 52. Ventana de configuración de la conexión de la placa.

9.4. Cuestiones sobre el ejercicio

¿De qué depende la velocidad de intermitencia del led? ¿Cómo se puede modificar? ¿Cómo se puede conseguir que la intermitencia del led se ajuste a un parámetro temporal de N segundos exactos e invariables?

¿Cómo modificar el programa para reflejar en los leds una cuenta binaria de 4 bits? ¿y si se quieren 8 bits de cuenta?

Modificar el código C para utilizar punteros en lugar de la macro que se emplea para escribir en el puerto paralelo de los leds.

10. Ejercicios Prácticos

Una vez concluido y verificado el diseño de un sistema de referencia sobre la placa DE1-SoC, se plantean una serie de ejercicios con la finalidad de llevar a la práctica los diferentes conceptos teóricos desarrollados. Otro objetivo que se pretende abordar es el manejo de la diferente documentación e información que tenemos a nuestra disposición. Antes de comenzar cabe recordar, que en PoliformaT, dentro del “Material de Prácticas” donde se ubica este documento se dispone de dos carpetas adicionales una de “Ejemplos” y otra de “Manuales” con el material necesario para la realización de estos ejercicios.

10.1. Ejercicio 1 – El manejo de puertos paralelos

Para comenzar crearemos un nuevo proyecto de aplicación en Eclipse que denominaremos `puertos_paralelos`. Cada proyecto de aplicación va asociado con un proyecto BSP, en este caso se utilizará el mismo proyecto BSP (o de placa) empleado anteriormente (`led_intermitente_bsp`). Como código fuente tomaremos los archivos de ejemplo que se encuentran disponible en PoliformaT con la denominación: `paralelos.c`. Una vez compilado el proyecto y corregidos los posibles errores, si los tuviese, se verificará su funcionamiento en la placa DE1-SoC.

Al ejecutar el software en la placa DE1-SoC, en sus displays 7-segmentos se mostrará un gusanillo que va moviéndose por ellos. Los pulsadores e interruptores servirán para cambiar el patrón de la visualización.

Observar el empleo de punteros para la lectura y escritura de datos en los puertos paralelo. Hay que recordar que los punteros no permiten eludir el paso por la cache de datos. Por ello, se ubican los periféricos en la región de periféricos definida en los parámetros del procesador Nios II. En el ejemplo anterior del led intermitente, en lugar de punteros se utilizaba una macro, que si realiza un *bypass* de la cache de datos.

¿Cómo se puede modificar la velocidad del patrón que se visualiza en los displays 7-segmentos? ¿Es exacta la velocidad o aproximada? ¿Qué pasa si todos los interruptores están abajo (a cero) y pulsamos un pulsador KEY? ¿Qué función tiene la línea de código: `while (*KEY_ptr);`? ¿Qué ocurre si la eliminamos?

10.2. Ejercicio 2 – El manejo de las Interrupciones

Crearemos un nuevo proyecto de aplicación en Eclipse que denominaremos `interrupciones`. Se utilizará el mismo proyecto BSP empleado anteriormente (`led_intermitente_bsp`). Como código fuente tomaremos los archivos de ejemplo que se encuentran disponibles en PoliformaT en la carpeta “Interrupciones” dentro de “Ejemplos”. Son un total de cuatro archivos que se deben incorporar al proyecto: `interrupt_example.c`, `interval_timer_ISR.c`, `pushbuttons_ISR.c` y `key_codes.h`. Una vez compilado el proyecto y corregidos los posibles errores, si los tuviese, se verificará su funcionamiento en la placa DE1-SoC.

En este ejemplo, se utilizan las interrupciones avisar al procesador cuando se ha pulsado un pulsador de la placa DE1-SoC y por otra parte para indicar cuando el temporizador ha llegado a su fin, generando, así como una especie de alarma de fin de temporización, pero periódica porque el temporizador se vuelve a cargar con el mismo valor automáticamente. La temporización se configura, por medio del uso de la IP del sistema “*Interval Timer*”, cuyo funcionamiento se encuentra explicado en el manual *Manual Perifericos IP.pdf*.

Al ejecutar el software en la placa DE1-SoC, en sus displays 7-segmentos se mostrará un gusanillo que va moviéndose por ellos. Los pulsadores servirán para cambiar la dirección del movimiento, y el patrón de la visualización. Mediante, la interrupción del temporizador, se controla la velocidad del movimiento.

¿Cómo se puede modificar la velocidad del patrón que se visualiza en los displays 7-segmentos? ¿Qué tenemos que cambiar para que la velocidad se ajuste a 1 segundo? ¿Es exacta la velocidad o aproximada? ¿Qué pasa si todos los interruptores están abajo (a cero) y pulsamos el KEY3? ¿Qué pasa si pulsamos KEY0? ¿Por qué?

10.3. Ejercicio 3 – Visualización en la pantalla VGA

Crearemos un nuevo proyecto de aplicación en Eclipse que denominaremos `test_vga`, con su proyecto BSP para la placa denominado: `test_vga_bsp`. Como código fuente tomaremos los archivos de ejemplo que se encuentran disponible en PoliformaT con la denominación: `test_vga.c`. Antes de realizar la compilación del proyecto hay que realizar una modificación de los parámetros del *linker*, ya que se va a utilizar la memoria SDRAM tanto para el programa como para memoria de vídeo. Se procederá de la siguiente forma:

1. Haciendo clic en el botón derecho del ratón sobre el proyecto `test_vga_bsp`, seleccionar **Nios II > BSP Editor…**. Esto abrirá la ventana del editor del BSP.
2. Seleccionar en la nueva ventana, la pestaña denominada **Linker Script**. Ver *Figura 53*.
3. En el recuadro intermedio denominado **Linker Memory Regions**, se selecciona la *sdram* y se elimina. Se crearán regiones nuevas con el botón **Add…**, de la derecha.

4. Crear una nueva región de memoria denominada *sdram*, con un tamaño de 65011680 y un offset de 32, tal como se ve en la *Figura 54*.
5. Crear una segunda región de memoria denominada *vsdram*, con un tamaño de 2097152 y un offset de 65011712, tal como se ve en la *Figura 54*.
6. Pulsar el botón **Generate** para creación de los parámetros y salir.
7. Ya se puede compilar el proyecto.

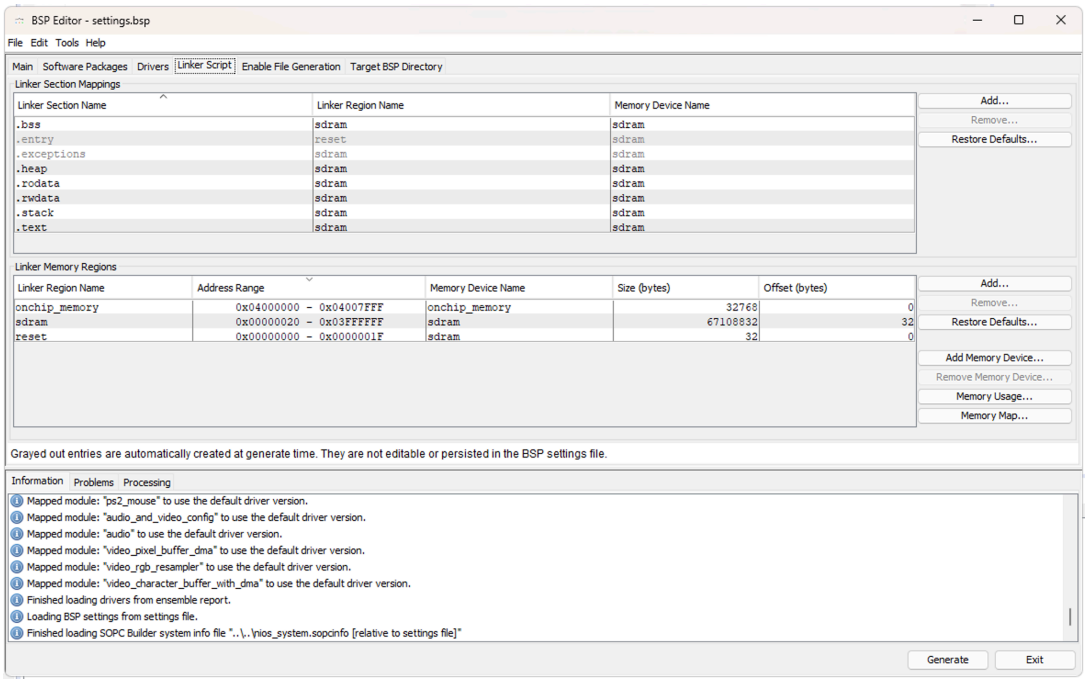

Figura 53. Pestaña inicial Linker Script del Editor de BSP.

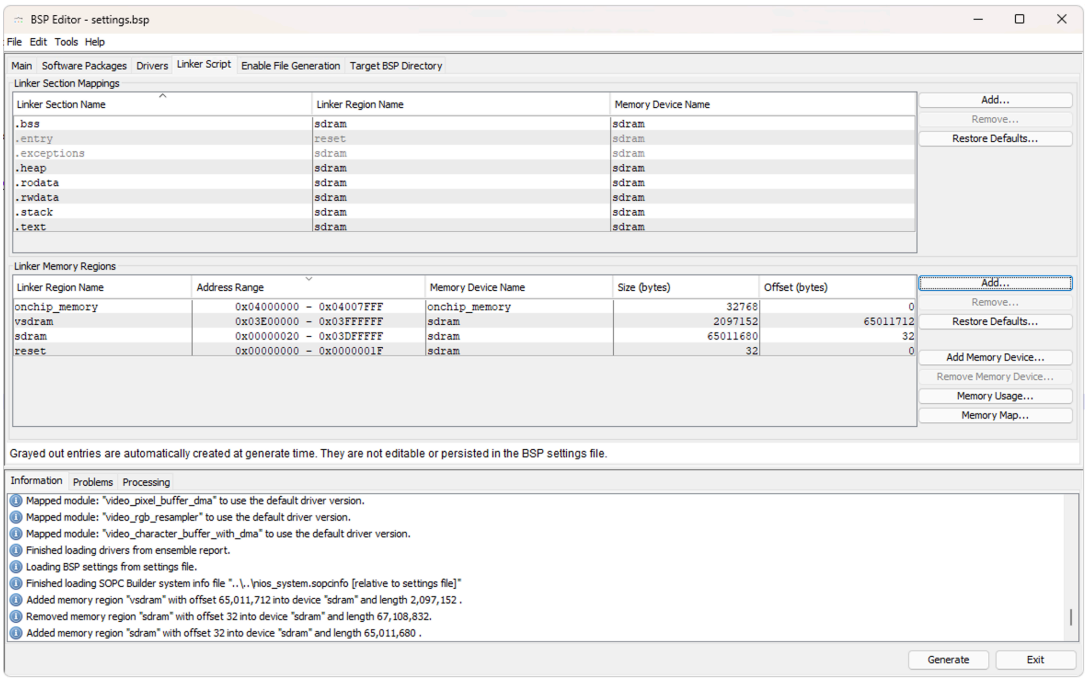

Figura 54. Pestaña modificada Linker Script del Editor de BSP.

Una vez compilado el proyecto y corregidos los posibles errores, si los tuviese, se verificará su funcionamiento en la placa DE1-SoC. Cuando el hardware comience la ejecución del programa, en la pestaña **Nios II Console** del Nios II SBT se mostrará el mensaje de salida `Hola desde Nios II!`. Mientras que en la placa DE1-SoC, los displays 7-segmentos representarán unos segmentos en movimiento y al conmutar los interruptores se encenderán los LED. El centro de la pantalla VGA aparecerá un mensaje dentro de un recuadro azul.

En la subrutina de dibujar un cuadrado en la pantalla **`VGA_box`**, aparece la siguiente línea de código: **`offset = (row << 9) + col;`** ¿Que función realiza esta línea de código? ¿A qué se debe ese 9? ¿Se puede modificar el 9 por otro valor? ¿Cuál sería el efecto?

Modifica el código, para que el color del recuadro central dependa del valor de los interruptores, según la siguiente tabla:

Tabla 1. Selección del Color.

SW[2]	SW[1]	SW[0]	Color
0	0	0	Negro
0	0	1	Azul
0	1	0	Verde
0	1	1	Cian
1	0	0	Rojo
1	0	1	Magenta
1	1	0	Amarillo
1	1	1	Blanco

10.4. Ejercicio 4 – Diseño de un reloj

Realizar un programa software en C, para el sistema hardware anteriormente diseñado, que implemente un reloj. El reloj debe mostrar la hora, los minutos y los segundos en los displays 7-segmentos de la placa DE1-SoC, utilizando el formato de 24 horas.

El reloj debe funcionar en tiempo real, viéndose como se van incrementando los segundos, los minutos y las horas con el paso del tiempo. Como base para el funcionamiento del reloj se tomará el Temporizador o “Interval Timer”, que dispone el sistema hardware anteriormente creado. Dicho temporizador se encuentra configurado para realizar una interrupción periódica al sistema cada 125 ms, con lo cual se puede utilizar dicha interrupción para incrementar una cuenta que a su vez permita incrementar los segundos, los minutos y las horas de manera correcta.

El temporizador del sistema puede ser configurado por software, para que en lugar de generar interrupciones cada 1 ms, las realice con la cadencia que se desee; por ejemplo 100 ms. Para ello hay que tener presente que la señal de reloj que utiliza el temporizador para su funcionamiento es de 100 MHz.

Para poder ajustar la hora del reloj a la hora actual, se utilizarán los pulsadores (KEY3 a KEY0) de la placa DE1-SoC, de tal manera que al pulsar el KEY3 se incrementen en una unidad las horas y al pulsar KEY2 se incrementen en una unidad los minutos. La gestión de los pulsadores se debe realizar también mediante interrupciones; es decir, que al pulsar uno de los pulsadores se genere una interrupción que, al ser atendida por el microprocesador, realice una lectura de los pulsadores para determinar cuál de ellos ha sido pulsado y actuar en consecuencia, incrementando las horas o los minutos.

Además de visualizar la hora, minutos y segundos en el display 7-segmentos, también debe hacerse en la pantalla VGA. Así mismo deberá aparecer en la pantalla VGA el nombre de los autores del ejercicio. Como la pantalla VGA, permite el uso de gráficos y texto, se valorará positivamente la creatividad en la visualización de la hora en la pantalla VGA, así como el uso de animaciones, texto, cambio de colores, etc.

Por último, opcionalmente se puede completar el ejercicio con el uso del teclado PS2, para funciones de ajuste de la hora o de la visualización.

10.5. Tarea 1 – Entregables

La valoración de esta tarea requiere de la entrega de archivos y documentos que permitan valoran las competencias y objetivos alcanzados en la realización de los ejercicios propuestos. La entrega se realizará exclusivamente por medio de las "Tareas" del PoliformaT de la asignatura. En ningún caso se aceptarán entregar por medios alternativos, como correos electrónicos, espacios compartidos o servicios en la nube.

La entrega deberá estar formada por los siguientes archivos:

- **Memoria Descriptiva.** Archivo que debe ser nombrado como ***Memoria_Tarea01.pdf***. Se trata de un documento en formato PDF que deberá incluir lo siguiente:
  - Portada con el nombre del alumno o alumnos que han realizado la Tarea.
  - Estructura del contenido del archivo de resolución, explicando cada fichero que es y que contiene.
  - Respuesta a las preguntas planteadas en la realización de los ejercicios de este documento.
  - Resolución de los ejercicios planteados, explicando y justificando las modificaciones realizadas y con comentarios a los aspectos más destacables.
  - Referencias utilizadas. Si en la resolución de los ejercicios se utiliza código proveniente de fuentes externas, debe referenciarse para evitar conflictos de valoración.

- **Archivo de resolución**. Archivo que se nombrará como ***Resolucion_Tarea01.zip***. Archivo en formato comprimido zip que incluirá los siguientes archivos y carpetas:
  - Archivo ***Leeme.txt***, explicando la estructura y contenido del archivo comprimido.
  - Archivo ***DE1_SoC_Media_Computer.sof***, para la configuración del hardware en la FPGA.
  - Una subcarpeta por cada ejercicio, nombrada ***Ejercicio_num*** donde ***num*** es el número del ejercicio, que contendrá los siguientes archivos:
    - Archivo ***Ejercicio_num.elf***, para la ejecución del software en la placa.
    - Archivos con el código C/C++ de resolución del ejercicio correspondiente. Los archivos de C/C++ deberán contener una cabecera con el nombre del autor o autores.

Importante, debido a la complejidad de corrección de los ejercicios y el elevado número de tareas a corregir, el incumplimiento de alguno de los requerimientos de entrega indicados anteriormente implicará la no corrección de la Tarea. De manera extraordinaria se podrá solicitar una reentrega de la Tarea.